Linux System Programming: A Complete 40-Day Mastery Guide

From Zero to Expert — For Absolute Beginners

> *"In the beginning, there was the command line."*
> — Neal Stephenson

📋 PREREQUISITES CHECKLIST

Before you begin this journey, ensure you have:

- [ ] A computer (Windows, Mac, or Linux)
- [ ] At least 20GB free disk space
- [ ] Internet connection
- [ ] Curiosity and patience!
- [ ] A virtual machine (VMware/VirtualBox) OR dual-boot Linux OR WSL2 on Windows
- [ ] Recommended: Ubuntu 22.04 LTS or Fedora 38+

No prior programming experience required. This guide starts from absolute zero.

⚡ QUICK START GUIDE (5 Minutes)

Get Linux running RIGHT NOW:

Option A: Windows (Fastest)

```
# Enable WSL2 in PowerShell (Run as Administrator)
wsl --install
# Restart your computer, then open Ubuntu from Start Menu
```

Option B: Online (Zero Install)

Visit https://www.katacoda.com or https://replit.com for instant Linux terminals.

Option C: VirtualBox

1. Download VirtualBox from virtualbox.org
2. Download Ubuntu 22.04 ISO from ubuntu.com
3. Create new VM → 2GB RAM → 20GB disk → Install Ubuntu

Verify your setup:

```
# Type this in your terminal - if it works, you're ready!
echo "Hello, Linux World!"
uname -a
```

📅 40-DAY LEARNING ROADMAP

```
WEEK 1 (Days 1-7):   Linux Foundations
  Day 1-2:  What is Linux? Installation & First Steps
  Day 3-4:  Files, Directories, and Navigation
  Day 5-6:  Permissions and Users
  Day 7:    Review + Mini-Project

WEEK 2 (Days 8-14):  Terminal Mastery
  Day 8-9:  Terminal Deep Dive & Processes
  Day 10-11: File Manipulation & Text Processing
  Day 12-13: Shell Scripting Basics
  Day 14:   Review + Mini-Project

WEEK 3 (Days 15-21): System Management
  Day 15-16: Package Management
  Day 17-18: System Services & systemd
  Day 19-20: Networking Basics
  Day 21:   Review + Mini-Project

WEEK 4 (Days 22-28): Advanced Topics
  Day 22-23: Process Scheduling & Priorities
  Day 24-25: Kernel Concepts
  Day 26-27: System Monitoring & Performance
  Day 28:   Review + Mini-Project

WEEK 5 (Days 29-35): Expert Level
  Day 29-30: Advanced Shell Scripting
  Day 31-32: Security & Hardening
  Day 33-34: Automation & Cron
  Day 35:   Review + Mini-Project
```

```
WEEK 6 (Days 36-40): Mastery & Projects
  Day 36-37: Final Projects
  Day 38-39: Interview Prep
  Day 40:   Certification Prep & Next Steps
```

PART 1: LINUX SYSTEMS

Section 1.1: Introduction to Linux System Software

🎯 Learning Objectives

- Understand what Linux is and why it matters
- Identify the components of a Linux distribution
- Distinguish between the kernel, shell, and user space
- Compare Linux to other operating systems
- Install and boot a Linux system

Introduction

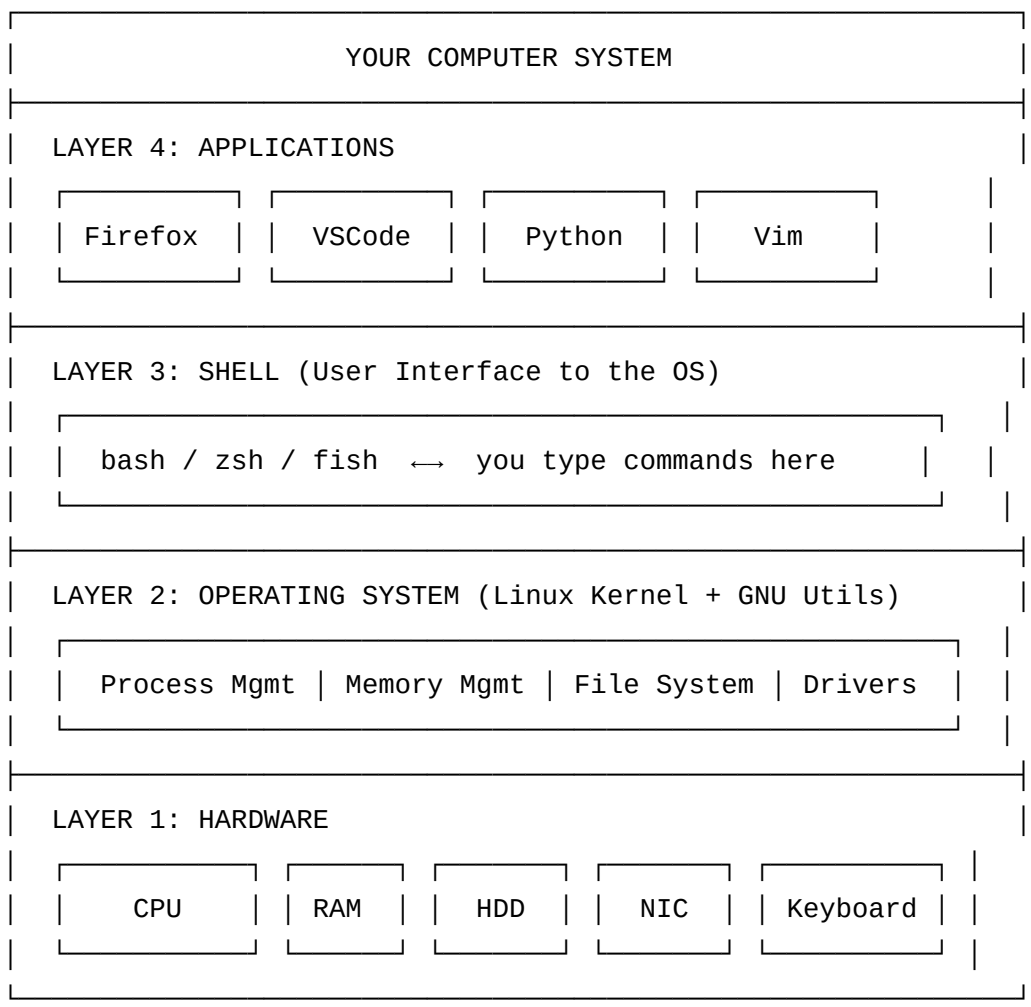
Imagine you own a restaurant. The building itself — the walls, plumbing, electrical — is like the **kernel**. The kitchen equipment is the **system**

utilities. The menus and recipes are **applications**. And the chef who takes your order and delivers food is the **shell** — your interface to everything.

Linux is not just an operating system. It's a philosophy. It's a community. And it's the engine powering the world's infrastructure: 96.3% of the top 1 million web servers run Linux. 100% of the world's top 500 supercomputers run Linux. Your Android phone? Linux underneath. The cloud? Mostly Linux.

Linux was created by Linus Torvalds in 1991, as a free alternative to expensive UNIX systems. What started as a student project became the backbone of the modern internet.

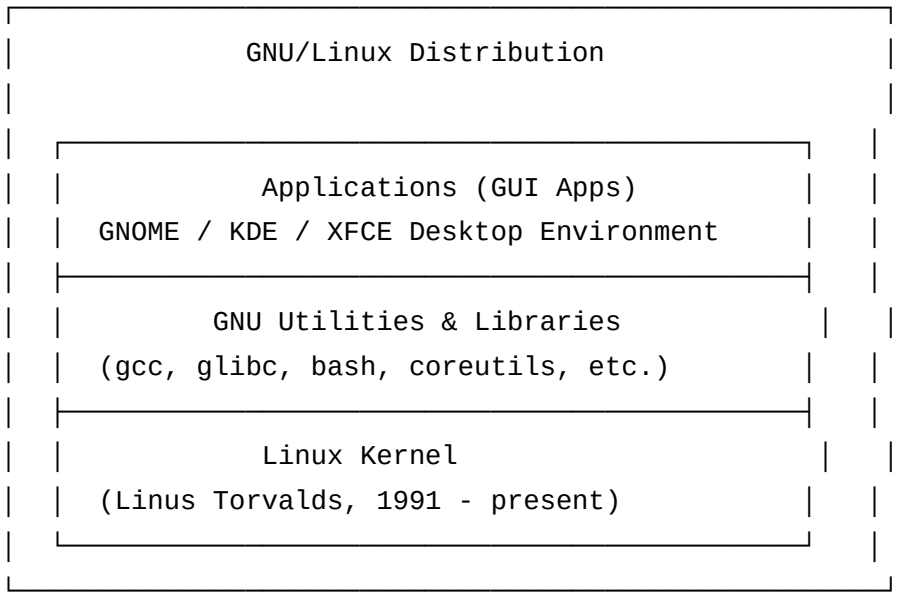
What is an Operating System?



An **Operating System (OS)** is software that manages computer hardware and provides services for programs. Think of it as the manager of a busy office — it ensures everyone gets resources, no one hogs the printer, and files are organized properly.

Linux as an Operating System

Linux is technically just the **kernel** (the core of the OS). What most people call "Linux" is actually a complete operating system combining:



Components of a Linux Distribution

A **Linux Distribution (Distro)** is a complete, ready-to-use Linux system. Different distros make different choices about which software to include and how to configure things.

COMPONENT	WHAT IT IS	EXAMPLES
Linux Kernel	Core OS — manages hardware	5.15, 6.1, 6.5
Init System	First process, starts services	systemd, SysVinit
GNU Core Utils	Basic commands (ls, cp, mv)	GNU Coreutils
Shell	Command interpreter	bash, zsh, fish
Package Manager	Install/remove software	apt, dnf, pacman
Desktop Env	Graphical interface	GNOME, KDE, XFCE
Display Server	Manages windows & graphics	X11, Wayland
System Libraries	Shared code for programs	glibc, libstdc++

Popular Linux Distributions Comparison

DISTRO	BASE	PKG MGR	DESKTOP	BEST FOR
Ubuntu	Debian	apt	GNOME	Beginners, Desktop
Debian	-	apt	Multiple	Servers, Stability
Fedora	RPM	dnf	GNOME	Developers, Latest
CentOS/RHEL	RPM	yum/dnf	Multiple	Enterprise Servers
Arch Linux	-	pacman	DIY	Experts, Learning
Linux Mint	Ubuntu	apt	Cinnamon	Windows switchers
Kali Linux	Debian	apt	GNOME	Security/Pentesting
Alpine	-	apk	None	Containers, IoT

🏭 Real-World Use Case

Netflix uses Ubuntu for its CDN servers that deliver streaming video. **Google** uses a custom Linux distribution called "gLinux" on employee workstations. **Android** (3+ billion devices) uses the Linux kernel at its core.

Practical Examples

Example 1: Checking Your Linux Distribution

```
# Find out which distro you're running
cat /etc/os-release

# Expected output (Ubuntu example):
# NAME="Ubuntu"
# VERSION="22.04.3 LTS (Jammy Jellyfish)"
# ID=ubuntu
# PRETTY_NAME="Ubuntu 22.04.3 LTS"

# Short version
lsb_release -a

# Just the distro name
cat /etc/issue
```

Example 2: Checking the Kernel Version

```
# See the Linux kernel version
uname -r

# Example output: 5.15.0-91-generic

# Full system information
uname -a

# Example output:
# Linux hostname 5.15.0-91-generic #101-Ubuntu SMP Tue Nov 14 13:30:08 UTC 2023
# x86_64 x86_64 x86_64 GNU/Linux
```

Example 3: Exploring System Information

```
# See CPU information
cat /proc/cpuinfo | grep "model name" | head -1
# Output: model name : Intel(R) Core(TM) i7-10700 CPU @ 2.90GHz

# See memory information
free -h
# Output:
#               total    used    free    shared  buff/cache   available
# Mem:           15Gi    4.2Gi   8.1Gi   456Mi    3.1Gi        10Gi
# Swap:         2.0Gi    0.0Bi   2.0Gi

# See disk usage
df -h
```

Example 4: The Linux File System Overview

```
# See the root directory structure
ls /

# Key directories:
# /bin    - Essential commands (ls, cp, mv)
# /etc    - Configuration files
# /home   - User home directories
# /var    - Variable data (logs, databases)
# /tmp    - Temporary files
# /usr    - User programs and data
# /proc   - Virtual filesystem for process info
# /sys    - Virtual filesystem for hardware info
# /dev    - Device files
```

Example 5: Getting Help

```
# Get help for any command with man (manual)
man ls

# Shorter help
ls --help

# Info pages (more detailed)
info bash

# Which command is this?
which ls
# Output: /usr/bin/ls

# All instances of a command
whereis ls
```

🔧 Hands-on Exercise 1.1

Goal: Explore your Linux system and answer these questions.

```
# Run each command and record what you find:

# 1. What distribution are you running?
cat /etc/os-release | grep PRETTY_NAME

# 2. What kernel version?
uname -r

# 3. How much RAM do you have?
free -h | grep Mem

# 4. How much disk space?
df -h /

# 5. What is your username?
whoami

# 6. What is your home directory?
echo $HOME

# 7. What shell are you using?
echo $SHELL
```

Expected outputs will vary by system. The goal is to run the commands and understand what they show.

⚠️ Common Mistakes

1. **"Linux is too hard"** — Start with Ubuntu. The modern desktop is user-friendly.
2. **Confusing Linux with Unix** — Linux is inspired by Unix but is distinct and free.

3. **Thinking there's only one Linux** — There are 500+ distros. Ubuntu is just the most popular.
4. **Running everything as root** — Never run as root unless necessary. Use `sudo` for specific commands.

✅ Best Practices

- **Start with Ubuntu or Linux Mint** if you're a beginner
- **Use a VM first** before installing on real hardware
- **Read error messages** — Linux error messages are informative
- **Use tab completion** — Press Tab to auto-complete commands
- **Keep a terminal reference** handy for the first few weeks

📚 Quick Reference — Linux Distributions

```
BEGINNER FRIENDLY:
  Ubuntu 22.04 LTS  →  ubuntu.com
  Linux Mint 21     →  linuxmint.com

SERVER FOCUSED:
  Ubuntu Server     →  ubuntu.com/server
  Debian            →  debian.org
  CentOS Stream     →  centos.org

DEVELOPER FOCUSED:
  Fedora            →  getfedora.org
  Arch Linux        →  archlinux.org

SECURITY FOCUSED:
  Kali Linux        →  kali.org
  Parrot OS         →  parrotsec.org
```

🔗 Additional Resources

- **Official Ubuntu Documentation:** help.ubuntu.com
- **Linux Journey (interactive):** linuxjourney.com
- **The Linux Command Line (book):** nostarch.com/tlcl2
- **Distrowatch (compare distros):** distrowatch.com

Section 1.2: Linux Kernel Overview

🎯 Learning Objectives

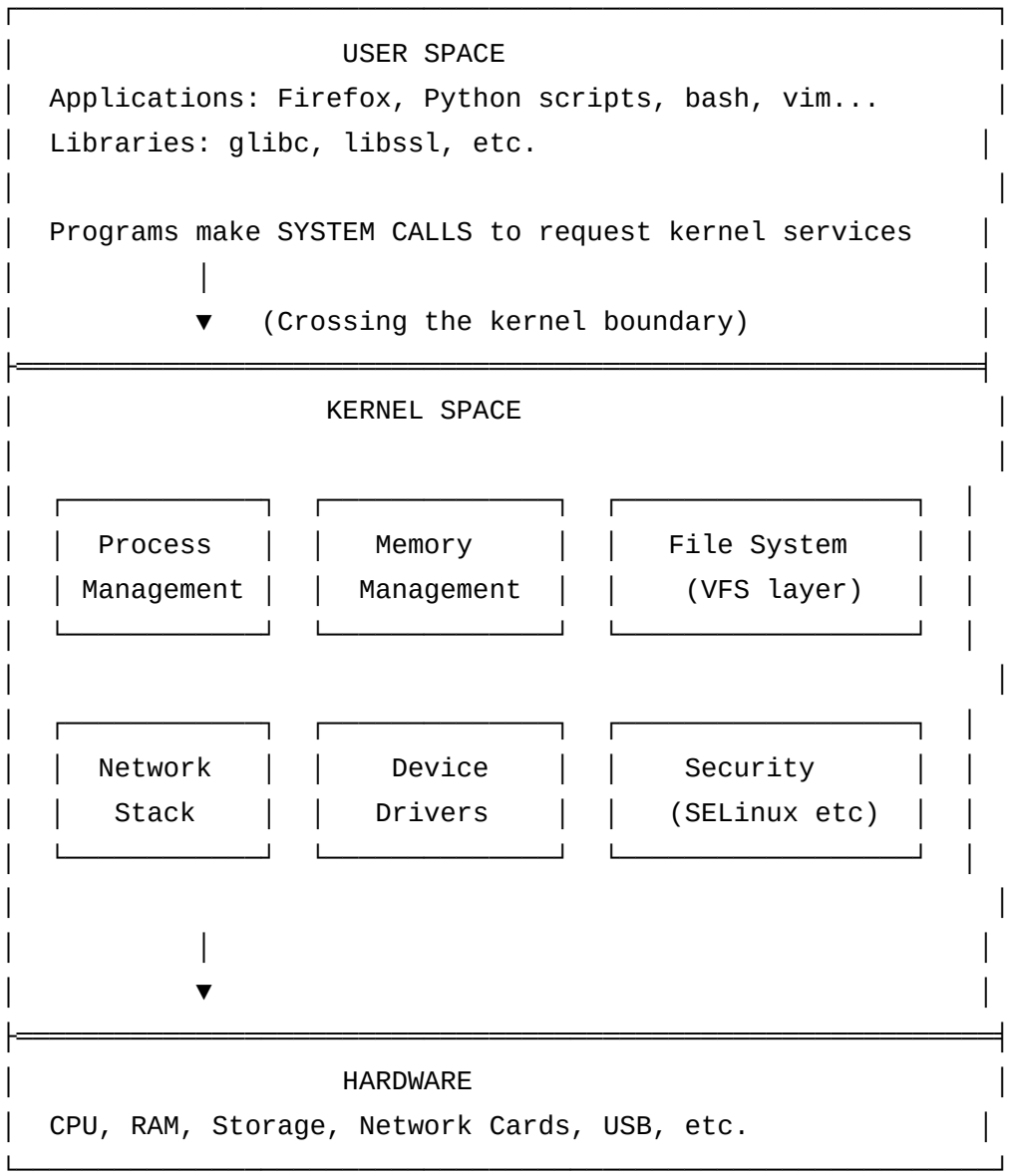
- Explain what the Linux kernel does and why it exists
- Understand the difference between kernel space and user space
- Read and interpret kernel version numbers
- Explain the kernel's role in resource management
- Identify key kernel subsystems

Introduction

If your computer were a country, the kernel would be the government — setting rules, allocating resources, maintaining order, and providing services that everyone depends on. Programs don't directly touch the hardware; they ask the kernel, and the kernel decides whether and how to fulfill the request.

The Linux kernel is the result of decades of collaborative work by thousands of developers worldwide. It's arguably the most complex piece of software ever created, with over 30 million lines of code, yet it boots in seconds and runs for years without a restart.

Role of the Kernel



The kernel's primary responsibilities:

1. Process Management — Creates, schedules, and terminates processes. Decides which process gets CPU time and when.

2. Memory Management — Allocates and deallocates RAM. Manages virtual memory and prevents processes from accessing each other's memory.

3. Device Management — Talks to hardware through device drivers. Abstracts hardware so programs don't need to know hardware details.

4. File System — Organizes storage into files and directories. Supports dozens of file system formats (ext4, XFS, NTFS, FAT32).

5. Network Management — Implements TCP/IP and other protocols. Routes packets, manages sockets.

6. Security — Enforces permissions, capabilities, and mandatory access control.

Linux Kernel Versions

```
VERSION FORMAT:  Major.Minor.Patch-Extra
EXAMPLE:         6.1.55-1-MANJARO
                 | |  |   └── Distribution-specific suffix
                 | |  └────── Patch level (bug fixes)
                 | └───────── Minor version (new features)
                 └─────────── Major version (rare changes)
```

Kernel Version History (Key Milestones)

YEAR	VERSION	NOTABLE FEATURE
1991	0.01	First public release by Linus Torvalds
1994	1.0	First stable production kernel
1996	2.0	SMP support (multiple processors)
1999	2.2	Better networking
2001	2.4	USB, DevFS, better memory management
2003	2.6	Completely redesigned scheduler
2011	3.0	Just a number change (continued from 2.6)
2015	4.0	Live kernel patching
2019	5.0	Just a number change (continued from 4.x)
2022	5.15	LTS version with lots of hardware support
2022	6.0	New features, ARM64 improvements
2023	6.5	Latest stable (as of late 2023)

How to Check Kernel Version

```
# Current kernel version
uname -r
# 5.15.0-91-generic

# Detailed kernel info
uname -a
# Linux hostname 5.15.0-91-generic #101-Ubuntu SMP ...

# Alternative method
cat /proc/version
# Linux version 5.15.0-91-generic (buildd@lcy02-amd64-028)
# (gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0, ...)
# #101-Ubuntu SMP Tue Nov 14 13:30:08 UTC 2023

# See installed kernel packages (Ubuntu/Debian)
dpkg --list | grep linux-image

# See all available kernels
ls /boot/vmlinuz*
```

Long-Term Support (LTS) vs Rolling Release

SUPPORT TYPE	EXAMPLE	KERNEL UPDATES	STABILITY
LTS Kernel	5.15, 6.1	Security fixes only	Very High
Latest Stable	6.5	New features	High
Rolling	Arch Linux	Latest always	Medium
Development	linux-next	Unstable features	Low

💡 **Pro Tip:** For servers, always use LTS kernels. For desktop learning, use the latest stable to get new hardware support.

Practical Examples

Example 1: Exploring the /proc Filesystem (Window into the Kernel)

```
# The /proc filesystem is NOT real storage - it's the kernel showing you live data!

# See running processes
ls /proc/
# You'll see numbered directories (1, 2, 3...) = process IDs

# Info about a specific process (PID 1 = init/systemd)
cat /proc/1/status

# CPU info from the kernel
cat /proc/cpuinfo

# Memory usage from the kernel
cat /proc/meminfo

# System uptime (in seconds)
cat /proc/uptime
# 1234.56 4567.89
# (time since boot) (idle time)

# Loaded kernel modules
cat /proc/modules | head -20
# or easier:
lsmod | head -20
```

Example 2: Kernel Modules

```
# Kernel modules are plugins that extend kernel functionality
# They can be loaded/unloaded without rebooting!

# List loaded modules
lsmod
# Module                  Size  Used by
# tcp_bbr                24576  0
# xt_MASQUERADE          20480  1
# ...

# Get info about a module
modinfo tcp_bbr

# Load a module (needs sudo)
sudo modprobe tcp_bbr

# Remove a module
sudo modprobe -r tcp_bbr
```

Example 3: Kernel Ring Buffer (System Log)

```
# The kernel keeps a log of events — very useful for debugging!

# See the kernel message buffer
dmesg

# See the last 20 lines
dmesg | tail -20

# Watch for new kernel messages in real time
dmesg -w

# Find USB-related messages
dmesg | grep -i usb

# Find error messages
dmesg | grep -i error

# See with timestamps (human readable)
dmesg -T
```

Example 4: Kernel System Calls

```
# System calls are how user programs talk to the kernel
# The 'strace' command lets you SPY on what system calls a program makes!

# Install strace if needed
sudo apt install strace  # Ubuntu
sudo dnf install strace  # Fedora

# See all system calls made by 'ls'
strace ls /tmp 2>&1 | head -30

# Count system calls by type
strace -c ls /tmp 2>&1

# This shows you: read, write, open, close, stat, etc.
# These are all requests to the kernel
```

🔧 Hands-on Exercise 1.2

```
# Exercise: Explore the kernel through /proc

# 1. Find your kernel version
uname -r

# 2. How long has the system been running?
uptime

# 3. Check memory info (find MemTotal, MemFree)
grep -E "MemTotal|MemFree" /proc/meminfo

# 4. Find out how many CPU cores you have
grep -c "processor" /proc/cpuinfo

# 5. See the first 5 kernel boot messages
dmesg | head -5

# 6. List 5 loaded kernel modules
lsmod | head -6

# CHALLENGE: Find out your system's load average
cat /proc/loadavg
# Numbers: 1-min avg, 5-min avg, 15-min avg, running/total, last PID
```

⚠️ Common Mistakes

1. **Thinking the kernel is the whole OS** — The kernel is just one part; distros add much more.
2. **Compiling your own kernel as a beginner** — Unnecessary; distros provide optimized kernels.

3. **Manually loading/unloading modules without knowing what they do** — Can destabilize the system.
4. **Ignoring kernel updates** — Security patches are critical; keep your kernel updated.

✅ Best Practices

- **Update your kernel regularly** for security patches: `sudo apt update && sudo apt upgrade`
- **Use LTS kernels on servers** for stability
- **Monitor dmesg** when hardware problems occur — it's your first diagnostic tool
- **Never run custom kernel code from untrusted sources** — the kernel has full hardware access

Section 1.3: Kernel Architecture & Components

🎯 Learning Objectives

- Describe the monolithic vs microkernel architecture
- Identify all major kernel subsystems and their purpose
- Understand how kernel modules extend functionality
- Explain the virtual file system (VFS) abstraction
- Navigate kernel documentation

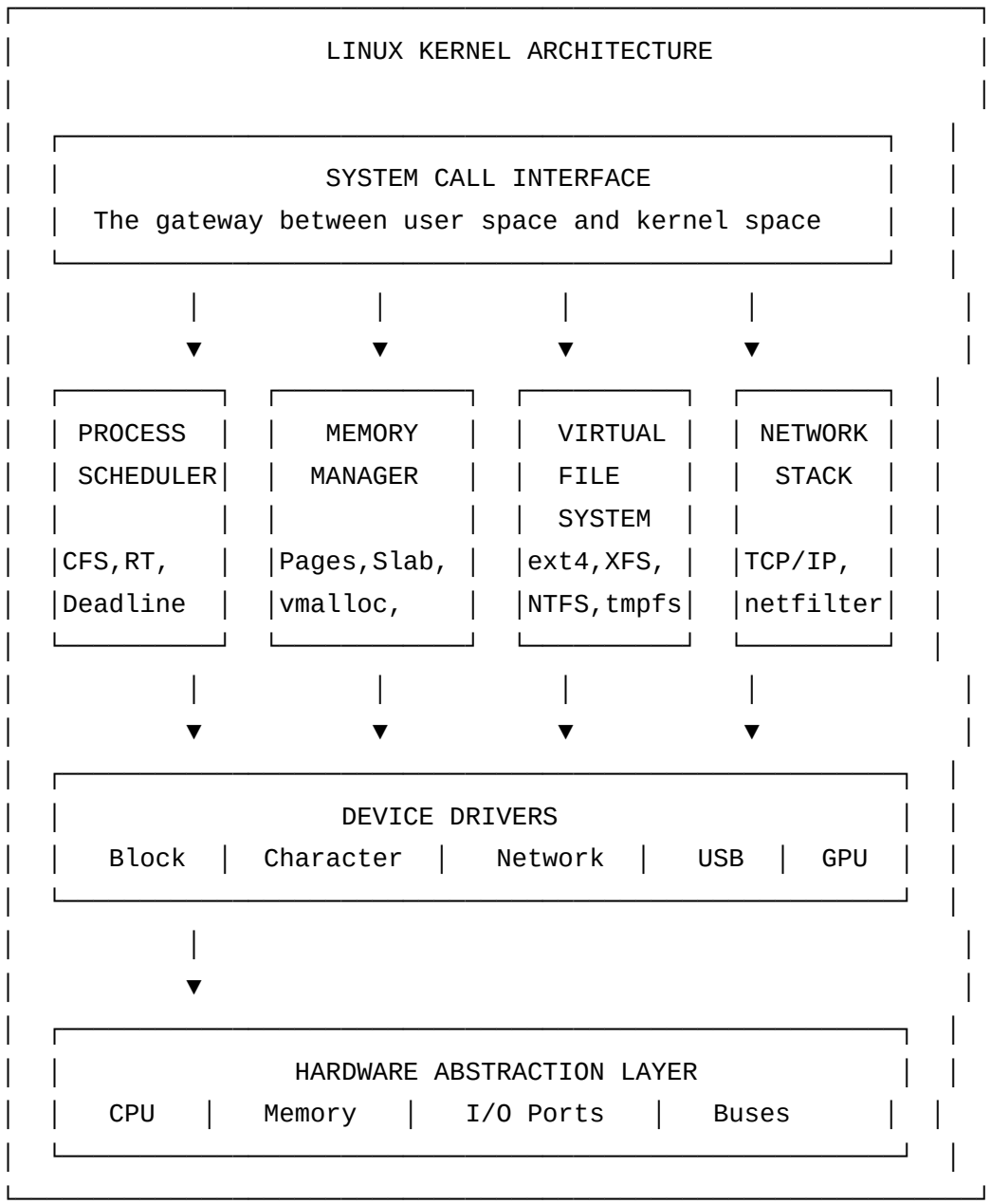
Introduction

The Linux kernel follows a **monolithic architecture** — all kernel services run in the same privileged memory space, making it fast but meaning

that a buggy driver can potentially crash the whole system. This is in contrast to a **microkernel** (like GNU Hurd), where services run in separate processes for safety, but with performance overhead.

Linux compensates for the risk of monolithic design through rigorous code review, kernel modules (which can be loaded/unloaded safely), and memory protection between drivers.

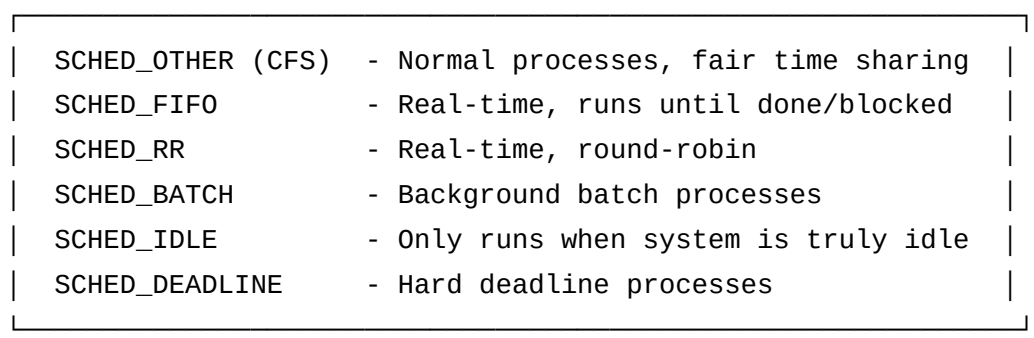
Understanding the Kernel's Structure


LINUX KERNEL ARCHITECTURE
SYSTEM CALL INTERFACE
The gateway between user space and kernel space
PROCESS SCHEDULER
CFS,RT, Deadline
MEMORY MANAGER
Pages,Slab, vmalloc,
VIRTUAL FILE SYSTEM
ext4,XFS, NTFS,tmpfs
NETWORK STACK
TCP/IP, netfilter
DEVICE DRIVERS
Block
Character
Network
USB
GPU
HARDWARE ABSTRACTION LAYER
CPU
Memory
I/O Ports
Buses

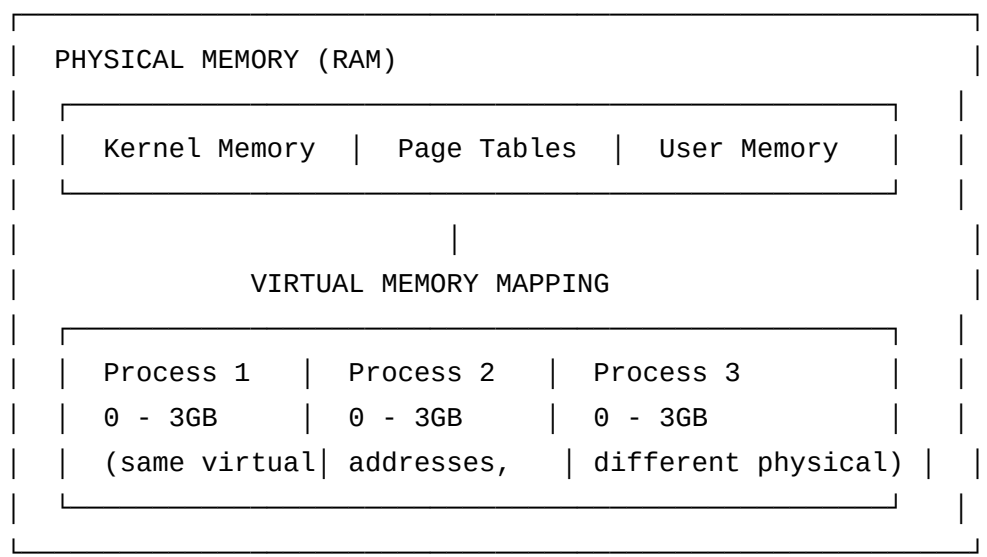
Kernel Components and Their Functions

1. Process Scheduler

The scheduler decides which process runs on which CPU core and for how long. Linux uses the **Completely Fair Scheduler (CFS)** as default.

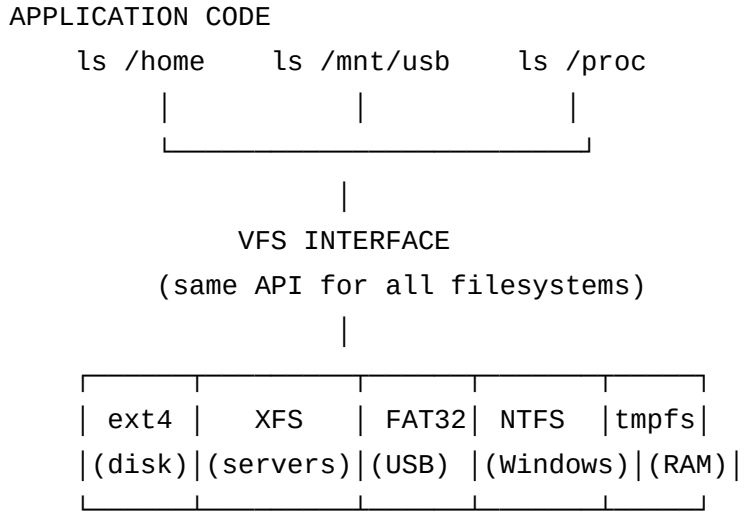


2. Memory Manager

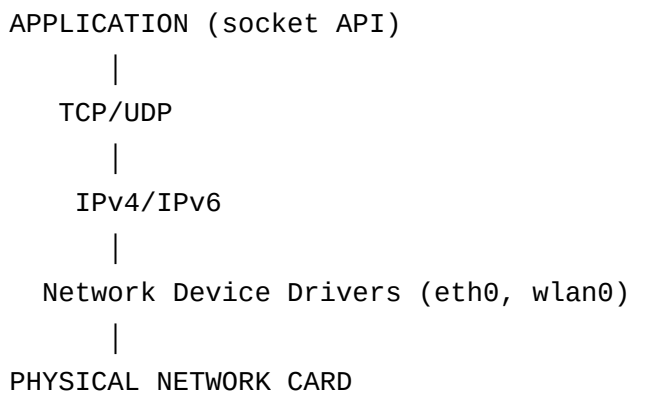


3. Virtual File System (VFS)

The VFS is a brilliant abstraction layer that lets you use one set of commands (open, read, write, close) with ANY file system:



4. Network Stack



Monolithic vs Microkernel Comparison

FEATURE	MONOLITHIC (Linux)	MICROKERNEL
Architecture	All in kernel space	Minimal kernel only
Performance	Very fast	Slower (IPC overhead)
Stability	One bad driver = crash	Isolated crashes
Complexity	Complex kernel	Simple kernel
Size	Large (30M+ lines)	Small
Examples	Linux, Windows	GNU Hurd, MINIX
Development	Loadable modules	Separate servers
Memory Usage	More kernel memory	More system overhead

Practical Examples

Example 1: Examining Kernel Subsystems

```
# See memory zones and pages
cat /proc/zoneinfo | head -30

# See file system types kernel supports
cat /proc/filesystems
# Shows all VFS-registered file systems

# See mounted filesystems
mount | column -t

# Or better:
findmnt

# Network connections in kernel
cat /proc/net/tcp
```

Example 2: Kernel Parameters (sysctl)

```
# View ALL kernel parameters
sysctl -a 2>/dev/null | head -30

# Common kernel parameters:
# vm.swappiness       - How aggressively to swap (0-100)
# net.ipv4.ip_forward - Enable routing between interfaces
# kernel.hostname     - System hostname
# kernel.pid_max      - Maximum process ID

# See a specific parameter
sysctl vm.swappiness
# vm.swappiness = 60

# Change a parameter temporarily (resets on reboot)
sudo sysctl -w vm.swappiness=10

# Make permanent (add to /etc/sysctl.conf)
echo "vm.swappiness = 10" | sudo tee -a /etc/sysctl.conf
```

Example 3: Kernel Memory Information

```
# Detailed memory stats
cat /proc/meminfo

# Key values:
# MemTotal:     16262272 kB  (total RAM)
# MemFree:       8234116 kB  (completely free)
# MemAvailable: 11902328 kB  (available for apps)
# Buffers:        234532 kB  (disk read buffer)
# Cached:        3291076 kB  (file cache)
# SwapTotal:     2097148 kB  (swap space)
# SwapFree:      2097148 kB  (free swap)

# Memory in human-readable format
free -h

# Slab allocator (kernel's internal memory)
cat /proc/slabinfo | head -20
```

Example 4: Kernel Modules Deep Dive

```
# See all loaded modules with info
lsmod

# Detailed info about a module
modinfo bluetooth
# Shows: filename, license, description, author, depends, etc.

# See module dependencies
modinfo bluetooth | grep depends

# Load module and all its dependencies
sudo modprobe bluetooth

# Find where modules are stored
ls /lib/modules/$(uname -r)/kernel/

# Find a module by name
find /lib/modules/$(uname -r) -name "*.ko" | grep bluetooth

# Create a dependency database (usually automatic)
sudo depmod -a
```

🔧 Hands-on Exercise 1.3

```
# Explore the Kernel Architecture

# 1. How many kernel parameters are there?
sysctl -a 2>/dev/null | wc -l

# 2. What file systems does your kernel support?
cat /proc/filesystems

# 3. What is your current swappiness value?
sysctl vm.swappiness

# 4. How many kernel modules are loaded?
lsmod | wc -l

# 5. What is MemAvailable? (available RAM for applications)
grep MemAvailable /proc/meminfo

# 6. Find the size of the kernel itself
ls -lh /boot/vmlinuz-$(uname -r)

# CHALLENGE: Trace a system call
strace echo "hello" 2>&1 | grep -E "^write"
```

Section 1.4: File and Directory Operations

🎯 Learning Objectives

- Navigate the Linux file system hierarchy confidently
- Create, move, copy, and delete files and directories
- Understand absolute vs relative paths
- Use wildcards and globbing effectively
- Work with hidden files and special directories

Introduction

In Linux, **everything is a file**. Not just documents and images — your keyboard is a file (`/dev/input/keyboard`), your network connection is a file (a socket), and even information about running processes is presented as files (`/proc/1234/status`). This "everything is a file" philosophy makes Linux incredibly consistent and powerful.

Understanding the file system is your most fundamental Linux skill. Every other operation builds on this foundation.

Linux Directory Structure (Filesystem Hierarchy Standard)

```
/ (root — the top of everything)
├── bin/        Essential user commands (ls, cp, mv, bash)
├── boot/       Boot files (kernel, bootloader)
├── dev/        Device files (disks, terminals, etc.)
├── etc/        System configuration files
├── home/       User home directories
│   ├── alice/
│   └── bob/
├── lib/        Essential shared libraries
├── lib64/      64-bit shared libraries
├── media/      Mount points for removable media
├── mnt/        Temporary mount points
├── opt/        Optional/third-party software
├── proc/       Virtual filesystem (process/kernel info)
├── root/       Root user's home directory
├── run/        Runtime data (PID files, sockets)
├── sbin/       System binaries (for root/admin)
├── srv/        Service data (web server files, etc.)
├── sys/        Virtual filesystem (hardware info)
├── tmp/        Temporary files (cleared on reboot)
├── usr/        User programs, documentation, libraries
│   ├── bin/    Non-essential user commands
│   ├── lib/    Non-essential libraries
│   ├── local/  Locally installed software
│   └── share/  Architecture-independent data
└── var/        Variable data (logs, databases, mail)
    ├── log/    System log files
    ├── mail/   Mail spool
    └── spool/  Print/cron queues
```

Navigating the File System

Absolute vs Relative Paths

```
ABSOLUTE PATH: Starts from root (/)
  /home/alice/documents/report.txt

RELATIVE PATH: Starts from current directory
  documents/report.txt  (if you're in /home/alice)

SPECIAL SHORTCUTS:
  .    = Current directory
  ..   = Parent directory
  ~    = Your home directory (/home/alice)
  -    = Previous directory (like browser back button)

EXAMPLE:
  Current location: /home/alice/documents

  cd ..          → goes to /home/alice
  cd ../..       → goes to /home
  cd ~           → goes to /home/alice (always)
  cd ~/music     → goes to /home/alice/music
  cd -           → goes back to /home/alice/documents
```

Essential File and Directory Operations

Navigation Commands

```
# Print Working Directory (where am I?)
pwd
# Output: /home/alice

# List files
ls              # Simple list
ls -l           # Long format (permissions, size, date)
ls -la          # Include hidden files
ls -lh          # Human-readable file sizes
ls -lt          # Sort by modification time
ls -lS          # Sort by file size
ls -R           # Recursive (all subdirectories)

# Change Directory
cd /etc                    # Go to /etc
cd ~                       # Go home
cd ..                      # Go up one level
cd -                       # Go to previous directory
cd /var/log/nginx          # Full path navigation
```

Creating Files and Directories

```
# Create empty file
touch filename.txt
touch file1.txt file2.txt file3.txt  # Multiple files at once

# Create directory
mkdir mydir
mkdir -p path/to/nested/dir  # Create all intermediate dirs

# Create file with content
echo "Hello, World!" > hello.txt     # Creates/overwrites
echo "More text" >> hello.txt        # Appends
printf "Line 1\nLine 2\n" > output.txt

# Create file with heredoc
cat > myfile.txt << 'EOF'
This is line 1
This is line 2
This is line 3
EOF
```

Copying Files

```
# Copy file
cp source.txt destination.txt
cp source.txt /tmp/               # Copy to directory
cp file1.txt file2.txt /tmp/      # Copy multiple files

# Copy directory (must use -r for recursive)
cp -r sourcedir/ destdir/
cp -r /etc/nginx /tmp/nginx-backup

# Copy with preserved metadata
cp -p source.txt destination.txt  # Preserve timestamps/perms
cp -a sourcedir/ destdir/         # Archive mode (best for backup)

# Interactive (ask before overwrite)
cp -i source.txt destination.txt
```

Moving and Renaming

```
# Move file (also used for renaming!)
mv oldname.txt newname.txt         # Rename
mv file.txt /tmp/                  # Move to directory
mv *.txt /tmp/                     # Move all .txt files

# Move directory
mv olddir/ newdir/

# Interactive (ask before overwrite)
mv -i source.txt destination.txt
```

Deleting Files

```
# Remove file
rm file.txt
rm file1.txt file2.txt file3.txt  # Multiple files

# Remove directory
rmdir emptydir/           # Only works on empty directories
rm -r directory/          # Recursive (for non-empty dirs)
rm -rf directory/         # Force (no prompts) - BE CAREFUL!

# ⚠️ WARNING: rm -rf with wrong path = disaster!
# There is no trash can in the terminal!
# SAFE PRACTICE: Use trash-cli instead
sudo apt install trash-cli
trash file.txt            # Move to trash (recoverable)
```

Viewing Files

```
# Display entire file
cat file.txt

# Display with line numbers
cat -n file.txt

# View large files (one page at a time)
less file.txt
# Navigation in less:
#   Space/F = next page
#   B = previous page
#   G = end of file
#   1G = beginning
#   /pattern = search
#   q = quit

# First N lines
head file.txt          # Default: 10 lines
head -n 20 file.txt    # First 20 lines
head -20 file.txt      # Also first 20 lines

# Last N lines
tail file.txt          # Default: 10 lines
tail -n 20 file.txt    # Last 20 lines
tail -f file.txt       # Follow file (great for logs!)
tail -f /var/log/syslog  # Watch syslog in real time
```

Wildcards and Globbing

WILDCARD	MEANING	EXAMPLE	MATCHES
*	Zero or more chars	*.txt	a.txt, hello.txt
?	Exactly one char	file?.txt	file1.txt, fileA.txt
[abc]	One of these chars	file[123].txt	file1.txt, file2.txt
[a-z]	Range	[A-Z]*.txt	Hello.txt, World.txt
[!abc]	NOT these chars	file[!0-9].txt	filea.txt (not file1.txt)
{a,b,c}	Any of these patterns	file.{txt,pdf}	file.txt, file.pdf

```
# Practical wildcard examples:

# List all .txt files
ls *.txt

# List files starting with 'log' and ending with digits
ls log[0-9]*

# Delete all .tmp files
rm *.tmp

# Copy all images
cp *.{jpg,png,gif} /tmp/images/

# Find files with exactly 4-character names
ls ????.txt

# Create many files at once
touch file_{1..10}.txt   # Creates file_1.txt through file_10.txt

# Remove files 5-8
rm file_{5..8}.txt
```

Practical File Operation Examples

Example 1: Complete Navigation Exercise

```
# Start from home
cd ~

# Create a project structure
mkdir -p projects/myapp/{src,tests,docs,config}

# Verify the structure
find projects/ -type d

# Output:
# projects/
# projects/myapp
# projects/myapp/src
# projects/myapp/tests
# projects/myapp/docs
# projects/myapp/config

# Navigate around
cd projects/myapp
pwd
# /home/alice/projects/myapp

cd src
pwd
# /home/alice/projects/myapp/src

cd ../../config
pwd
# /home/alice/projects/myapp/config
```

```
# Back to start quickly
cd ~
```

Example 2: File Organization Script

```
# Organize a messy directory by file type

# Create a demo messy directory
mkdir -p /tmp/messy
touch /tmp/messy/{a,b,c}.txt /tmp/messy/{x,y}.jpg /tmp/messy/z.pdf

# Organize it
cd /tmp/messy
mkdir -p organized/{text,images,pdfs}

# Move by type
mv *.txt organized/text/
mv *.jpg organized/images/
mv *.pdf organized/pdfs/

# Verify
find organized/ -type f
```

Example 3: Working with Hidden Files

```
# Hidden files start with a dot (.)
# Configuration files are usually hidden

# See hidden files
ls -la ~

# Key hidden files in home directory:
# .bashrc          - bash configuration
# .bash_history    - command history
# .ssh/            - SSH keys and config
# .config/         - Application configs
# .local/          - Local user data

# Create a hidden file
echo "secret config" > ~/.myconfig

# See it
ls -la ~ | grep myconfig

# Don't delete hidden system files by accident!
# ⚠️ rm -rf ~/.[!.]* could delete important configs
```

Example 4: Finding Files

```
# find is the most powerful file search tool

# Find by name
find / -name "passwd" 2>/dev/null
find ~ -name "*.txt"

# Find by type
find /tmp -type f    # Files only
find /etc -type d    # Directories only
find /dev -type l    # Symlinks only

# Find by size
find ~ -size +100M   # Larger than 100MB
find /tmp -size -1k  # Smaller than 1KB

# Find by time
find ~ -mtime -7     # Modified in last 7 days
find /var/log -mtime +30  # Not modified in 30+ days

# Find and execute command
find /tmp -name "*.tmp" -delete       # Find and delete
find ~ -name "*.txt" -exec wc -l {} \;  # Count lines in each

# Locate (uses database, faster but not always current)
locate filename.txt
sudo updatedb   # Update the database
```

Example 5: Links (Shortcuts)

```
# HARD LINK: Another name for the same file
ln original.txt hardlink.txt
# Both point to same data. Deleting original doesn't delete data.

# SYMBOLIC LINK (SYMLINK): A shortcut/alias
ln -s /usr/bin/python3 ~/mypy
# Like a Windows shortcut. Deleting original = broken link.

# Practical symlink example:
# You installed Python 3.11 but scripts expect 'python'
sudo ln -s /usr/bin/python3 /usr/local/bin/python

# See symlinks
ls -la /usr/bin/python*
ls -la /etc/alternatives/ | head -10
```

🔧 Hands-on Exercise 1.4

```
# EXERCISE: Build and Navigate a Directory Structure

# Step 1: Create this exact structure in your home directory:
# ~/workshop/
# ├── project1/
# │   ├── src/
# │   ├── tests/
# │   └── README.txt
# ├── project2/
# │   ├── src/
# │   └── README.txt
# └── shared/
#     └── utils.txt

# Create the structure
mkdir -p ~/workshop/project1/{src,tests}
mkdir -p ~/workshop/project2/src
mkdir -p ~/workshop/shared

# Create the files
echo "# Project 1" > ~/workshop/project1/README.txt
echo "# Project 2" > ~/workshop/project2/README.txt
echo "# Shared Utils" > ~/workshop/shared/utils.txt

# Step 2: Navigation challenges
# Q1: Go to project1/src and come back in ONE command
cd ~/workshop/project1/src && cd -

# Q2: List all .txt files in workshop recursively
find ~/workshop -name "*.txt"
```

```
# Q3: Copy README.txt from project1 to project2
cp ~/workshop/project1/README.txt ~/workshop/project2/README-copy.txt

# Q4: How many files are in workshop total?
find ~/workshop -type f | wc -l

# Step 3: Cleanup
rm -rf ~/workshop
echo "Done! Workshop cleaned up."
```

⚠️ Common Mistakes

1. **`rm -rf /`** — The most catastrophic command. Always double-check your path before running `rm -rf`.
2. **Forgetting `-r` for directory operations** — `cp dir/` fails without `-r`, use `cp -r dir/`.
3. **Paths with spaces** — `cd My Documents` fails; use `cd "My Documents"` or `cd My\ Documents`.
4. **`ls` vs `ls -a`** — Confused why a directory seems empty? You might be missing hidden files!
5. **Relative vs absolute confusion** — Using relative paths in scripts that run from different directories.

✅ Best Practices

- **Always use `-i` flag with dangerous operations** in scripts: `rm -i`, `cp -i`, `mv -i`
- **Use `trash` instead of `rm`** for files you might need back
- **Quote variables** in scripts: `rm "$filename"` not `rm $filename`
- **Test with `echo` first**: `echo rm -rf /path/to/dir` to see what would be deleted
- **Use `find` before `rm`**: `find /path -name "*.tmp"` then `find /path -name "*.tmp" -delete`

📋 Quick Reference — File Operations Cheat Sheet

COMMAND	WHAT IT DOES	EXAMPLE
pwd	Show current directory	pwd
cd	Change directory	cd /etc
ls	List files	ls -lah
mkdir	Create directory	mkdir -p a/b/c
touch	Create empty file	touch file.txt
cp	Copy file/dir	cp -r src/ dst/
mv	Move/rename	mv old.txt new.txt
rm	Delete	rm -rf dir/
cat	View file	cat file.txt
less	Page through file	less bigfile.txt
head	First N lines	head -20 file.txt
tail	Last N lines	tail -f log.txt
find	Search for files	find ~ -name "*.txt"
ln	Create links	ln -s /path/to/file link

Section 1.5: Terminals and Controlling Processes

🎯 Learning Objectives

- Open, manage, and switch between multiple terminal sessions

- Understand TTY and pseudo-terminal (PTY) concepts
- Control foreground and background processes
- Use job control commands effectively
- Use tmux or screen for persistent sessions

Introduction

When you open a terminal window on Linux, you're not interacting directly with the kernel. You're using a **pseudo-terminal (PTY)** — a software layer that simulates an old-fashioned physical terminal (like a VT100 terminal from the 1970s). This might seem like unnecessary complexity, but it's what allows you to have multiple terminal windows, remote SSH sessions, and terminal multiplexers.

Understanding how terminals and processes interact is essential for working with Linux systems, especially when managing long-running processes on remote servers.

Terminals, TTYs, and PTYs

```
PHYSICAL TERMINAL (historical):
  Keyboard → Serial Port → TTY Device → Shell → CPU

MODERN VIRTUAL TERMINAL (text mode):
  Keyboard → Kernel TTY subsystem → /dev/ttyN → Shell

GRAPHICAL PSEUDO-TERMINAL:
  Terminal Emulator (gnome-terminal, xterm)
      ↕
  PTY Master/Slave Pair (/dev/pts/0, /dev/pts/1)
      ↕
  Shell (bash, zsh)

SSH REMOTE SESSION:
  Your Computer → Network → SSH Daemon → PTY → Shell


# See which terminal you're using
tty
# Output: /dev/pts/0

# See all active terminals
who
# alice    pts/0        2024-01-15 09:30 (:0)

# See your own terminal info
w
```

Virtual Consoles (Text Mode Terminals)

Linux provides **6 virtual consoles** accessible by default (Ctrl+Alt+F1 through F6). These are useful when the graphical interface crashes.

```
Ctrl+Alt+F1  = tty1 (often the GUI)
Ctrl+Alt+F2  = tty2 (virtual console 2)
Ctrl+Alt+F3  = tty3 (virtual console 3)
Ctrl+Alt+F4  = tty4
Ctrl+Alt+F5  = tty5
Ctrl+Alt+F6  = tty6
Ctrl+Alt+F7  = tty7 (usually back to GUI)
```

Process Control in the Terminal

Foreground and Background Jobs

```
FOREGROUND: Process that holds the terminal
  $ long-running-command
  (terminal is blocked until command finishes)

BACKGROUND: Process running without blocking terminal
  $ long-running-command &   (the & sends it to background)
  [1] 12345                  (job number and PID)
  $                          (prompt returns immediately!)
```

```
# Start a process in the background
sleep 60 &
# [1] 12345  (job 1, PID 12345)

# See background jobs
jobs
# [1]+  Running    sleep 60 &

# Bring job back to foreground
fg %1     # Bring job 1 to foreground
fg        # Bring last job to foreground

# Send foreground job to background
# First, suspend it with Ctrl+Z
sleep 60    # Press Ctrl+Z
# [1]+  Stopped    sleep 60
bg %1       # Resume in background

# Kill a background job
kill %1     # Send SIGTERM to job 1
kill %1 -9  # Force kill
```

Process Signals

```
SIGNAL NAME   NUMBER   MEANING
──────────────────────────────────────────────────────
SIGTERM         15    Polite "please stop" (default kill)
SIGKILL          9    "Die NOW" - cannot be caught/ignored
SIGHUP           1    "Terminal closed" / reload config
SIGINT           2    Interrupt (Ctrl+C)
SIGTSTP         20    Terminal stop (Ctrl+Z)
SIGCONT         18    Continue (after TSTP)
SIGCHLD         17    Child process ended
SIGUSR1         10    User-defined signal 1
SIGUSR2         12    User-defined signal 2


# Send signals to processes
kill 12345           # SIGTERM to PID 12345
kill -9 12345        # SIGKILL (force)
kill -HUP 12345      # Reload (often used for config reload)
kill -l              # List all signals

# Kill by name
killall firefox      # Kill all processes named 'firefox'
pkill -f "python"    # Kill processes matching pattern

# Interactive process killer
top                  # Press 'k', enter PID
htop                 # Click on process, press F9
```

The 3 Most Important Terminal Shortcuts

```
Ctrl+C  = SIGINT  = Interrupt/stop the running program
Ctrl+Z  = SIGTSTP = Pause/suspend the running program
Ctrl+D  = EOF     = End of input / logout
```

Terminal Multiplexers: tmux

`tmux` lets you have multiple terminal sessions in one window AND keep sessions running even after you disconnect (critical for remote server work!).

```
TMUX TERMINOLOGY:
  Session  = A collection of windows (like a workspace)
  Window   = Like a tab in your browser
  Pane     = Split areas within a window

BASIC TMUX USAGE:
  tmux                    # Start new session
  tmux new -s mysession     # Named session
  tmux ls                   # List sessions
  tmux attach -t mysession  # Re-attach to session
```

```
# Essential tmux commands (all start with Ctrl+B as prefix):
# Ctrl+B, ?        = Help / list all shortcuts
# Ctrl+B, c        = Create new window
# Ctrl+B, n        = Next window
# Ctrl+B, p        = Previous window
# Ctrl+B, 0-9      = Switch to window number
# Ctrl+B, d        = Detach from session (keeps running!)
# Ctrl+B, "        = Split pane horizontally
# Ctrl+B, %        = Split pane vertically
# Ctrl+B, arrows   = Move between panes
# Ctrl+B, z        = Zoom pane (full screen toggle)
# Ctrl+B, x        = Kill current pane
```

Practical tmux Workflow

```
# Start a new named session
tmux new -s work

# Create a window for logs
# Ctrl+B, c    (new window)
tail -f /var/log/syslog

# Create another window for commands
# Ctrl+B, c    (another new window)
# Type your commands here

# Split the current window to have two panes
# Ctrl+B, "    (horizontal split)
# Now you have two panes in one window

# Detach and go do something else
# Ctrl+B, d

# Come back later
tmux attach -t work

# The tail -f is still running!
```

Practical Examples

Example 1: Managing Long Downloads/Tasks

```
# Start a long task in background
wget https://ubuntu.com/ubuntu.iso -O /tmp/ubuntu.iso &
# [1] 54321

# Check progress
jobs
# [1]+  Running    wget ... &

# Bring to foreground to see progress
fg %1

# If you need to do something else, push back to background
# Ctrl+Z (pauses download!)
bg %1    # Resume in background

# Check wget process
ps aux | grep wget
```

Example 2: Keeping Processes Running After Logout

```
# Problem: You SSH to a server, start a script, and it dies when you disconnect
# Solution 1: nohup
nohup python3 myscript.py &
# Output goes to nohup.out
# Process continues after you log out!

# Solution 2: disown
python3 myscript.py &
disown %1
# Now it won't die when you disconnect

# Solution 3: tmux (best approach)
tmux new -s mywork
python3 myscript.py
# Ctrl+B, d  (detach - script keeps running)
# Log out
# Log back in
tmux attach -t mywork
# Your script is still running!
```

Example 3: Process Information

```
# See what terminal processes are using
ps aux
# USER  PID  %CPU %MEM   VSZ   RSS TTY   STAT  TIME COMMAND
# alice 1234  0.0  0.1  21776  5432 pts/0  Ss    0:00 bash
# alice 5678  0.0  0.0  37576  3548 pts/0  R+    0:00 ps aux

# TTY column:
# pts/0 = pseudo-terminal (terminal window)
# tty1  = virtual console 1
# ?     = no terminal (daemon/background process)

# Process tree
pstree -p
# Shows parent-child relationships
```

🔧 Hands-on Exercise 1.5

```
# Exercise: Job Control Practice

# 1. Start a long-running background process
sleep 300 &
echo "Started sleep 300 with PID: $!"

# 2. Start another one
sleep 400 &

# 3. Check your background jobs
jobs -l
# Should show 2 jobs with their PIDs

# 4. Bring the first job to foreground
fg %1
# You're now "in" sleep 300

# 5. Suspend it (Ctrl+Z)

# 6. Resume it in the background
bg %1

# 7. Kill job 2
kill %2

# 8. Wait for job 1 to die? No - kill it too
kill %1

# 9. Verify both are gone
jobs
```

```
# Should be empty

# BONUS: tmux practice
# Install tmux if needed: sudo apt install tmux
tmux new -s practice
# Create 3 windows (Ctrl+B, c)
# Split a window into panes (Ctrl+B, ")
# Detach (Ctrl+B, d)
tmux ls
tmux attach -t practice
```

Section 1.6: File Permissions

🎯 Learning Objectives

- Read and interpret the permission string (rwxrwxrwx)
- Modify permissions with chmod (symbolic and octal methods)
- Change file ownership with chown and chgrp
- Understand setuid, setgid, and sticky bit
- Apply appropriate permissions for security

Introduction

Linux permissions are a cornerstone of system security. Every file and directory has an owner, a group, and a set of permissions that control who

can read, write, or execute it. This system, while simple in concept, provides powerful access control for a multi-user system.

Imagine a library: you can read any book (read permission), only librarians can add/remove books (write permission), and anyone can use the computers (execute permission). Permissions work the same way for files.

Understanding Permission Structure

```
PERMISSION STRING FORMAT:
──────────────────────────────────────────────────────────────

 -  r  w  x    r  -  x    r  -  -
 |  |─────|    |─────|    |─────|
 |  Owner       Group       Others
 |
 File type:
   -  = Regular file
   d  = Directory
   l  = Symbolic link
   c  = Character device
   b  = Block device
   s  = Socket
   p  = Named pipe (FIFO)

PERMISSION MEANINGS:
  r (read)    = 4   Can read file content / list directory
  w (write)   = 2   Can modify file / create/delete in directory
  x (execute) = 1   Can run file as program / enter directory
  - (none)    = 0   No permission

OCTAL SYSTEM:
  rwx = 4+2+1 = 7
  rw- = 4+2+0 = 6
  r-x = 4+0+1 = 5
  r-- = 4+0+0 = 4
  -wx = 0+2+1 = 3
  -w- = 0+2+0 = 2
  --x = 0+0+1 = 1
```

```
--- = 0+0+0 = 0
```

Reading Permission Output

```
ls -la /etc/passwd
# -rw-r--r-- 1 root root 2468 Jan 15 10:30 /etc/passwd
#  |||||||||| | |    |    |    |              |
#  |||||||||| | |    |    |    |              └─ Filename
#  |||||||||| | |    |    |    └──────────────── Last modified
#  |||||||||| | |    |    └─────────────────────── Size (bytes)
#  |||||||||| | |    └──────────────────────────── Group owner
#  |||||||||| | └───────────────────────────────── User owner
#  |||||||||| └─────────────────────────────────── Hard link count
#  ||||||||└──────────────────────────────────────── Others: ---
#  |||||||└──────────────────────────────────────── Others: r
#  ||||||└──────────────────────────────────────── Others: r
#  |||||└──────────────────────────────────────── Group: --
#  ||||└──────────────────────────────────────── Group: r
#  |||└──────────────────────────────────────── Group: -
#  ||└──────────────────────────────────────── Owner: -
#  |└──────────────────────────────────────── Owner: w
#  └──────────────────────────────────────── Owner: r
#  (File type = -)
```

Managing File Permissions

chmod — Change Mode (Permissions)

Method 1: Symbolic (easier to understand)

```
# Syntax: chmod [who][+/-/=][permissions] file
# who: u=user/owner, g=group, o=others, a=all

# Add execute for owner
chmod u+x script.sh

# Remove write for others
chmod o-w sensitive.txt

# Set exact permissions for group (read and execute)
chmod g=rx program

# Add read for everyone
chmod a+r public_file.txt

# Multiple changes at once
chmod u+x,g-w,o-rwx file.txt

# Recursive (apply to directory and all contents)
chmod -R 755 /var/www/html/
```

Method 2: Octal (faster, used in scripts)

```
# chmod [octal] file
# Format: Owner-Group-Others (each is sum of r=4, w=2, x=1)

chmod 755 script.sh
# 7 = rwx (owner: read+write+execute)
# 5 = r-x (group: read+execute)
# 5 = r-x (others: read+execute)

chmod 644 config.txt
# 6 = rw- (owner: read+write)
# 4 = r-- (group: read only)
# 4 = r-- (others: read only)

chmod 600 private_key.pem
# 6 = rw- (owner: read+write)
# 0 = --- (group: nothing)
# 0 = --- (others: nothing)

chmod 777 public_writable/  # DANGEROUS! Everyone can do anything
chmod 000 hidden_file       # Nobody can do anything (even owner!)
```

Common Permission Patterns

PERMISSION	OCTAL	USE CASE
rwxr-xr-x	755	Executables, directories (public)
rw-r--r--	644	Regular files (public read)
rw-rw-r--	664	Collaborative files (group can write)
rw-------	600	Private files (owner only)
rwx------	700	Private scripts/directories
rwxrwxrwx	777	Fully public (AVOID on production)
r-xr-xr-x	555	Read-only executables
r--------	400	Read-only even for owner

chown — Change Owner

```
# Change owner
sudo chown alice file.txt

# Change owner and group
sudo chown alice:developers file.txt

# Change group only
sudo chown :developers file.txt
# Or use chgrp:
sudo chgrp developers file.txt

# Recursive
sudo chown -R www-data:www-data /var/www/html/

# Common use case: Fix permissions after extracting archive as root
sudo tar -xzf app.tar.gz
sudo chown -R alice:alice extracted_dir/
```

Special Permissions: setuid, setgid, sticky bit

```
SPECIAL BITS:
  setuid (s on user execute) = Program runs as file OWNER
  setgid (s on group execute) = Program runs as file GROUP
                                OR new files inherit directory group
  sticky bit (t on other execute) = Only owner can delete their files

EXAMPLES:
  -rwsr-xr-x  = setuid on executable
  -rwxr-sr-x  = setgid on executable
  drwxrwxrwt  = sticky bit on directory (/tmp uses this!)
```

```
# Check passwd command — it has setuid!
ls -la /usr/bin/passwd
# -rwsr-xr-x 1 root root 59976 ...
# The 's' means setuid — passwd runs as ROOT even when user runs it
# This lets regular users change their own password (stored in root-owned file)

# See /tmp — it has sticky bit
ls -la /tmp/..
# drwxrwxrwt  21 root root 4096 ...
# The 't' means sticky — anyone can create files, but only owner can delete

# Set special bits:
chmod u+s program         # Set setuid
chmod g+s directory/      # Set setgid
chmod +t /tmp/shared/     # Set sticky bit

# Octal: add 4000 for setuid, 2000 for setgid, 1000 for sticky
chmod 4755 program        # setuid + 755
chmod 2775 shared_dir/    # setgid + 775
chmod 1777 /tmp/shared/   # sticky + 777
```

Practical Examples

Example 1: Setting Up a Web Server Directory

```
# Web server (nginx/apache) runs as www-data user
# PHP files need to be readable by web server
# Config files should NOT be readable by web server

# Create web directory structure
sudo mkdir -p /var/www/mysite/{public,config,logs}

# Web-accessible files: www-data readable, but not writable
sudo chown -R alice:www-data /var/www/mysite/public/
sudo chmod -R 750 /var/www/mysite/public/

# Config files: only owner (alice) can read
sudo chown -R alice:alice /var/www/mysite/config/
sudo chmod -R 700 /var/www/mysite/config/

# Log files: web server needs to write
sudo chown -R www-data:www-data /var/www/mysite/logs/
sudo chmod -R 750 /var/www/mysite/logs/

# Verify
ls -la /var/www/mysite/
```

Example 2: Securing SSH Private Keys

```
# SSH keys MUST have correct permissions or SSH refuses to use them!

# Generate an SSH key pair
ssh-keygen -t ed25519 -C "your@email.com"
# Creates ~/.ssh/id_ed25519 (private) and ~/.ssh/id_ed25519.pub (public)

# Correct permissions
chmod 700 ~/.ssh/             # Only owner can access directory
chmod 600 ~/.ssh/id_ed25519   # Private key: owner read/write only
chmod 644 ~/.ssh/id_ed25519.pub  # Public key: world readable is fine

# Check your .ssh directory
ls -la ~/.ssh/

# If permissions are wrong, SSH will refuse to connect:
# WARNING: UNPROTECTED PRIVATE KEY FILE!
# Permissions 0777 for '/home/alice/.ssh/id_rsa' are too open.
```

Example 3: Creating a Shared Team Directory

```
# Scenario: /shared should be accessible by 'developers' group
# Anyone in the group should be able to create/edit files
# Files should automatically belong to 'developers' group

# Create group (if not exists)
sudo groupadd developers

# Add users to group
sudo usermod -aG developers alice
sudo usermod -aG developers bob

# Create shared directory
sudo mkdir /shared

# Set ownership and permissions
sudo chown root:developers /shared
sudo chmod 2775 /shared    # setgid (2) + rwxrwxr-x (775)
# The setgid bit means: new files inherit 'developers' group!

# Verify
ls -la /
# drwxrwsr-x  2 root developers 4096 Jan 15 ...

# Test: alice creates a file
su - alice
touch /shared/alice_file.txt
ls -la /shared/
# -rw-rw-r-- 1 alice developers 0 Jan 15 ... alice_file.txt
# Group is 'developers' automatically!
```

🔧 Hands-on Exercise 1.6

```
# Exercise: Permission Management

# Setup
mkdir -p /tmp/perm_test/{public,private,shared}
echo "public content" > /tmp/perm_test/public/index.html
echo "SECRET" > /tmp/perm_test/private/secret.key
echo "team file" > /tmp/perm_test/shared/teamwork.txt

# Task 1: Make public/index.html readable by everyone, writable only by owner
chmod 644 /tmp/perm_test/public/index.html

# Task 2: Make private/secret.key readable ONLY by owner
chmod 600 /tmp/perm_test/private/secret.key

# Task 3: Make public/ directory traversable by all, writable only by owner
chmod 755 /tmp/perm_test/public/

# Task 4: Make private/ directory accessible ONLY by owner
chmod 700 /tmp/perm_test/private/

# Verify your work
echo "=== public/ ==="
ls -la /tmp/perm_test/public/
echo "=== private/ ==="
ls -la /tmp/perm_test/private/

# ANSWERS:
# index.html should show: -rw-r--r--
# secret.key should show: -rw-------
# public/ should show:    drwxr-xr-x
```

```
# private/ should show:   drwx------

# Cleanup
rm -rf /tmp/perm_test/
```

Section 1.7: User and Group Management

🎯 Learning Objectives

- Create, modify, and delete user accounts
- Understand /etc/passwd, /etc/shadow, and /etc/group
- Manage groups and group membership
- Use sudo for privilege escalation
- Implement password policies

Introduction

Linux was designed from the ground up as a multi-user system. Even if you're the only person using your computer, you're interacting with a system that supports hundreds of simultaneous users. Understanding user and group management is essential for system administration, security, and setting up shared environments.

User Account Database Files

```
/etc/passwd — User account information (readable by all)
FORMAT: username:x:UID:GID:GECOS:home:shell

root:x:0:0:root:/root:/bin/bash
alice:x:1000:1000:Alice Smith:/home/alice:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin

FIELDS:
  username  = Login name
  x         = Password placeholder (actual hash in /etc/shadow)
  UID       = User ID (0=root, 1-999=system, 1000+=regular users)
  GID       = Primary group ID
  GECOS     = Full name / comment field
  home      = Home directory path
  shell     = Login shell (/usr/sbin/nologin = no login allowed)
```

```
/etc/shadow — Password hashes (only root can read!)
FORMAT: username:hash:last_change:min:max:warn:inactive:expire

alice:$6$salt$hashedpassword...:19700:0:99999:7:::

FIELDS:
  username    = Login name
  hash        = Encrypted password ($6$ = SHA-512)
  last_change = Days since Jan 1, 1970 of last change
  min         = Minimum days between password changes
  max         = Maximum days before change required
  warn        = Days before warning about expiry
  inactive    = Days after expiry until account locked
  expire      = Account expiry date (days since Jan 1, 1970)


/etc/group — Group information
FORMAT: groupname:x:GID:members

root:x:0:
sudo:x:27:alice,bob
developers:x:1001:alice,charlie,dave
www-data:x:33:

FIELDS:
  groupname = Group name
  x         = Group password (rarely used)
  GID       = Group ID
  members   = Comma-separated list of supplementary members
```

User and Group Administration

Creating Users

```
# Add a user (interactive, sets up home dir etc)
sudo adduser alice
# Asks for password, name, etc. Creates /home/alice

# Add a user (non-interactive)
sudo useradd -m -s /bin/bash -c "Alice Smith" alice
# -m = create home directory
# -s = set shell
# -c = comment/full name

# Add system user (for services, no login)
sudo useradd --system --no-create-home --shell /usr/sbin/nologin myservice
# Used for web servers, database servers, etc.

# Set password
sudo passwd alice
# Prompts for new password

# See user's info
id alice
# uid=1001(alice) gid=1001(alice) groups=1001(alice),27(sudo)

getent passwd alice
# alice:x:1001:1001:Alice Smith:/home/alice:/bin/bash
```

Modifying Users

```
# Change user's full name
sudo usermod -c "Alice M. Smith" alice

# Change user's shell
sudo usermod -s /bin/zsh alice

# Change user's home directory (and move files)
sudo usermod -d /home/newalice -m alice

# Lock a user account (disable login)
sudo usermod -L alice
sudo passwd -l alice

# Unlock a user account
sudo usermod -U alice
sudo passwd -u alice

# Set account expiry date
sudo usermod -e 2024-12-31 alice

# Add user to additional group
sudo usermod -aG sudo alice      # Add to sudo group
sudo usermod -aG docker alice    # Add to docker group
# ⚠️ CRITICAL: Always use -aG (append), not just -G!
# -G alone REPLACES all groups!
```

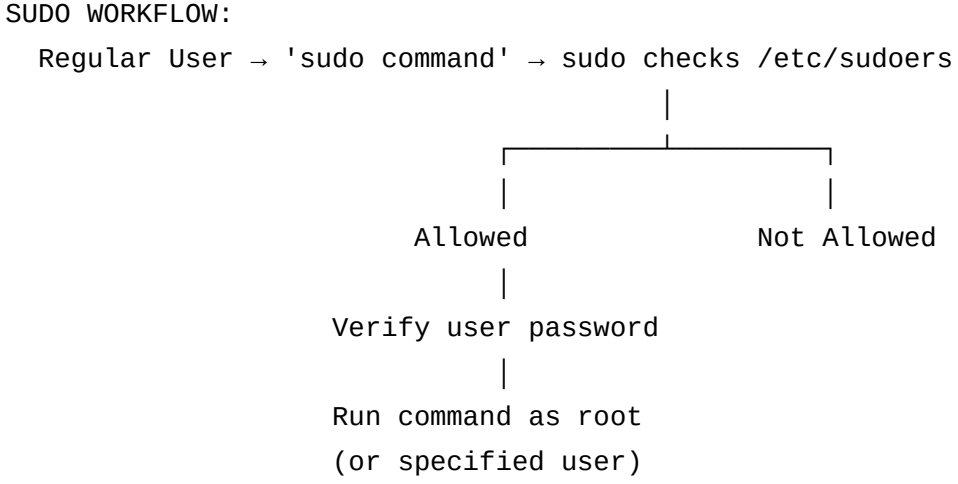
Deleting Users

```
# Remove user (keep home directory)
sudo userdel alice

# Remove user AND home directory AND mail spool
sudo userdel -r alice

# Check for files owned by user before deleting
find / -user alice 2>/dev/null
```

Managing Groups

```
# Create a group
sudo groupadd developers
sudo groupadd -g 2000 newgroup  # Specify GID

# Add user to group
sudo gpasswd -a alice developers  # Add
sudo gpasswd -d alice developers  # Remove

# Or with usermod:
sudo usermod -aG developers alice

# See group membership
groups alice
# alice : alice sudo developers

id alice
# uid=1001(alice) gid=1001(alice) groups=1001(alice),27(sudo),1002(developers)

# List all group members
getent group developers

# Modify group
sudo groupmod -n newname oldname  # Rename group

# Delete group
sudo groupdel developers
```

sudo — Controlled Privilege Escalation




```
# Run command as root
sudo apt update
sudo systemctl restart nginx

# Run command as another user
sudo -u www-data touch /var/www/file.txt

# Open root shell (use carefully!)
sudo -i         # Login shell
sudo -s         # Current environment
sudo su -       # Switch to root

# See your sudo privileges
sudo -l

# Edit sudoers safely (always use visudo!)
sudo visudo
```

sudoers file format:

```
# In /etc/sudoers (edit with: sudo visudo)

# Full sudo access for user
alice ALL=(ALL:ALL) ALL

# User can run specific commands without password
bob ALL=(ALL) NOPASSWD: /usr/bin/apt, /bin/systemctl restart nginx

# Group sudo access
%sudo ALL=(ALL:ALL) ALL
%developers ALL=(ALL) NOPASSWD: /usr/bin/systemctl

# FORMAT: user/group  hosts=(run_as_user:run_as_group)  commands
```

Practical Examples

Example 1: Setting Up a New Team Member

```
# New developer 'bob' joins the team

# 1. Create account
sudo adduser bob
# Enter password when prompted

# 2. Add to appropriate groups
sudo usermod -aG sudo bob         # Can use sudo
sudo usermod -aG developers bob   # Developer group
sudo usermod -aG docker bob       # Can use Docker

# 3. Set up SSH key access
sudo -u bob mkdir -p /home/bob/.ssh
sudo -u bob chmod 700 /home/bob/.ssh
# Add Bob's public key
echo "ssh-ed25519 AAAA...Bob's public key..." | sudo -u bob tee /home/bob/.ssh/authorized_keys
sudo -u bob chmod 600 /home/bob/.ssh/authorized_keys

# 4. Verify setup
id bob
groups bob
sudo -l -U bob
```

Example 2: Service Account Setup

```
# Create a dedicated user for a web application (never runs interactively)

sudo useradd \
  --system \
  --no-create-home \
  --shell /usr/sbin/nologin \
  --comment "MyApp Service Account" \
  myapp

# Create application directories
sudo mkdir -p /opt/myapp/{data,logs,config}
sudo chown -R myapp:myapp /opt/myapp/
sudo chmod 750 /opt/myapp/

# Verify — this account cannot be logged into
sudo -u myapp whoami   # Works (runs as myapp)
su - myapp             # Fails: This account is currently not available

# List all service accounts
getent passwd | grep nologin | cut -d: -f1
```

Example 3: Password Policy

```
# Set password expiry for a user
sudo chage -M 90 alice    # Max 90 days before change required
sudo chage -W 7 alice     # Warn 7 days before expiry
sudo chage -m 1 alice     # Must wait 1 day between changes

# See password aging info
sudo chage -l alice
# Last password change: Jan 15, 2024
# Password expires: Apr 14, 2024
# Password inactive: never
# Account expires: never
# Minimum number of days between password change: 1
# Maximum number of days between password change: 90
# Number of days of warning before password expires: 7

# Force password change on next login
sudo chage -d 0 alice
# or
sudo passwd -e alice
```

🔧 Hands-on Exercise 1.7

```
# Exercise: User and Group Management

# 1. Create a new user 'testuser'
sudo useradd -m -s /bin/bash -c "Test User" testuser
sudo passwd testuser

# 2. Create a group 'testers'
sudo groupadd testers

# 3. Add testuser to the testers group
sudo usermod -aG testers testuser

# 4. Verify testuser's groups
id testuser

# 5. See testuser in /etc/passwd
grep testuser /etc/passwd

# 6. See testuser's group in /etc/group
grep testers /etc/group

# 7. Create a shared directory for testers
sudo mkdir /tmp/testers_shared
sudo chown root:testers /tmp/testers_shared
sudo chmod 2775 /tmp/testers_shared

# 8. Test (switch to testuser)
sudo -u testuser touch /tmp/testers_shared/test.txt
ls -la /tmp/testers_shared/
```

```
# 9. Cleanup
sudo userdel -r testuser
sudo groupdel testers
sudo rm -rf /tmp/testers_shared
echo "Cleanup complete!"
```

Section 1.8: File Manipulation

🎯 Learning Objectives

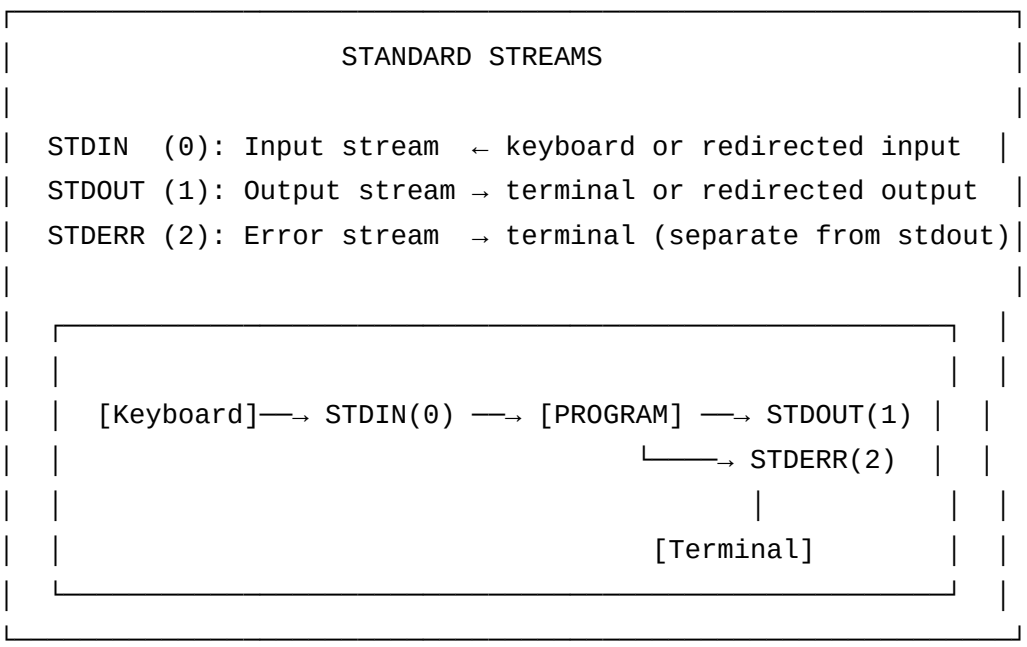
- Process text files with grep, sed, awk
- Master input/output redirection
- Use pipes to connect commands
- Search and transform file content
- Understand standard streams (stdin, stdout, stderr)

Introduction

Linux's power comes largely from the **Unix philosophy**: write programs that do one thing well, and combine them using pipes. The result is that complex data processing tasks can be accomplished with a single line of commands by chaining simple tools together.

A powerful one-liner like `ps aux | grep python | awk '{print $2}' | xargs kill` can terminate all Python processes — doing in seconds what would take minutes to write in a full program.

Standard Streams



Redirection and Piping

```
# OUTPUT REDIRECTION
command > file.txt      # Redirect stdout to file (overwrite)
command >> file.txt     # Redirect stdout to file (append)
command 2> error.txt    # Redirect stderr to file
command 2>&1            # Redirect stderr to stdout
command > out.txt 2>&1  # Both stdout and stderr to file
command &> out.txt      # Same as above (bash shorthand)
command > /dev/null     # Discard output (the black hole)
command > /dev/null 2>&1  # Discard ALL output

# INPUT REDIRECTION
command < file.txt      # Read stdin from file
command << 'EOF'        # Here document (inline input)
line 1
line 2
EOF

# PIPES (connect stdout of one to stdin of next)
command1 | command2           # Simple pipe
command1 | command2 | command3  # Chain
command1 |& command2          # Pipe both stdout AND stderr
```

Practical Redirection Examples

```
# Save command output to file
ls -la > filelist.txt
echo "Error occurred" >> error.log

# Redirect error messages to a log
./myscript.sh 2> errors.log

# Capture everything (stdout + stderr)
./myscript.sh > output.log 2>&1

# Discard errors (useful when you don't care about them)
find / -name "secret" 2>/dev/null

# Use file as input
sort < unsorted.txt

# Here document example (create file with multiple lines)
cat > config.txt << 'EOF'
server = localhost
port = 8080
debug = true
EOF
```

Text Processing Tools

grep — Search for Patterns

```
# Basic grep
grep "error" logfile.txt        # Find lines containing "error"
grep -i "error" logfile.txt     # Case-insensitive
grep -n "error" logfile.txt     # Show line numbers
grep -c "error" logfile.txt     # Count matching lines
grep -v "error" logfile.txt     # Invert (lines WITHOUT "error")
grep -r "TODO" ~/projects/       # Recursive search
grep -l "pattern" *.txt         # Show only filenames that match
grep -w "the" file.txt          # Match whole word only
grep -A 2 "error" log.txt       # Also show 2 lines AFTER match
grep -B 2 "error" log.txt       # Also show 2 lines BEFORE match
grep -C 2 "error" log.txt       # 2 lines before AND after

# Regular expressions
grep "^error" log.txt           # Line STARTS with "error"
grep "error$" log.txt           # Line ENDS with "error"
grep "err.r" log.txt            # "err" + any char + "r"
grep -E "error|warning" log.txt # Either "error" or "warning" (extended regex)
grep -P "\d{4}" data.txt        # Perl-compatible regex (4 digits)

# Practical examples
grep -r "def " ~/python_projects/ --include="*.py"  # Find all Python functions
grep -i "failed\|error" /var/log/syslog | tail -20  # Recent errors
```

sed — Stream Editor

```
# Basic substitution
sed 's/old/new/' file.txt        # Replace first occurrence per line
sed 's/old/new/g' file.txt       # Replace ALL occurrences
sed 's/old/new/I' file.txt       # Case-insensitive replacement
sed 's/old/new/2' file.txt       # Replace 2nd occurrence only

# Edit file IN PLACE
sed -i 's/old/new/g' file.txt    # Modify file directly
sed -i.bak 's/old/new/g' file.txt  # Modify but keep backup

# Delete lines
sed '5d' file.txt                # Delete line 5
sed '1,3d' file.txt              # Delete lines 1-3
sed '/pattern/d' file.txt        # Delete lines matching pattern

# Print specific lines
sed -n '5p' file.txt             # Print line 5 only
sed -n '1,10p' file.txt          # Print lines 1-10

# Insert/append
sed '2i\New line before line 2' file.txt  # Insert before line 2
sed '2a\New line after line 2' file.txt   # Append after line 2

# Practical examples:
# Replace localhost with actual server IP in config
sed -i 's/localhost/192.168.1.100/g' config.ini

# Remove blank lines
sed '/^$/d' file.txt
```

```
# Remove HTML tags
sed 's/<[^>]*>//g' index.html

# Add line numbers
sed = file.txt | sed 'N;s/\n/\t/'
```

awk — Pattern Scanning and Processing

```
# awk is a mini programming language for text processing

# Basic syntax: awk 'pattern { action }' file

# Print specific columns
awk '{print $1}' file.txt        # Print column 1
awk '{print $2, $4}' file.txt    # Print columns 2 and 4
awk '{print $NF}' file.txt       # Print last column (NF = number of fields)
awk '{print NR, $0}' file.txt    # Print line number + whole line

# Custom delimiters
awk -F: '{print $1}' /etc/passwd     # Use : as delimiter
awk -F, '{print $2}' data.csv        # Parse CSV

# Conditional processing
awk '$3 > 100 {print}' data.txt      # Print lines where column 3 > 100
awk '/error/ {print NR, $0}' log.txt # Print line numbers of error lines

# Math operations
awk '{sum += $3} END {print "Total:", sum}' sales.txt  # Sum column 3
awk '{count++} END {print count}' file.txt              # Count lines

# String operations
awk '{print toupper($0)}' file.txt   # Uppercase everything
awk '{gsub(/old/, "new"); print}' file.txt  # Global substitution

# Practical: Extract process info
ps aux | awk 'NR>1 {print $1, $2, $11}'  # user, PID, command
df -h | awk 'NR>1 {print $6, $5}'        # mount point, usage%
```

sort, uniq, wc — Statistics and Sorting

```
# sort
sort file.txt                    # Alphabetical sort
sort -r file.txt                 # Reverse sort
sort -n file.txt                 # Numeric sort
sort -k2 file.txt                # Sort by column 2
sort -k2 -n file.txt             # Sort by column 2, numeric
sort -t: -k3 -n /etc/passwd      # Sort passwd by UID

# uniq (must be sorted first for full effect!)
uniq file.txt                    # Remove consecutive duplicates
sort file.txt | uniq             # Remove all duplicates
sort file.txt | uniq -c          # Count occurrences
sort file.txt | uniq -d          # Show only duplicates
sort file.txt | uniq -u          # Show only unique lines

# wc — word count
wc file.txt                      # Lines, words, bytes
wc -l file.txt                   # Lines only
wc -w file.txt                   # Words only
wc -c file.txt                   # Bytes only
wc -m file.txt                   # Characters only

# Combined example: Find most common words in a file
cat file.txt | tr ' ' '\n' | sort | uniq -c | sort -rn | head -10
```

cut, tr, paste — Column Processing

```
# cut – extract columns
cut -d: -f1 /etc/passwd          # Field 1, colon-delimited
cut -d: -f1,7 /etc/passwd        # Fields 1 and 7
cut -c1-10 file.txt              # Characters 1-10

# tr – translate characters
echo "Hello World" | tr 'a-z' 'A-Z'   # Uppercase
echo "Hello World" | tr 'A-Z' 'a-z'   # Lowercase
echo "abc  def" | tr -s ' '            # Squeeze multiple spaces
echo "Hello\nWorld" | tr -d '\n'       # Delete newlines
echo "abc123" | tr -d '[:digit:]'      # Remove digits

# paste – merge files side by side
paste file1.txt file2.txt         # Merge columns
paste -d, file1.txt file2.txt     # Use comma as delimiter
```

Practical Complex Examples

Example 1: Log File Analysis Pipeline

```
# Scenario: Analyze Apache access log to find top 10 IP addresses

# Sample log format:
# 192.168.1.1 - - [15/Jan/2024:10:00:00 +0000] "GET /index.html HTTP/1.1" 200 1234

# Solution pipeline:
cat /var/log/apache2/access.log \
  | awk '{print $1}' \       # Extract IP address (column 1)
  | sort \                   # Sort IPs
  | uniq -c \                # Count occurrences
  | sort -rn \               # Sort by count, descending
  | head -10                 # Show top 10

# Output:
#   5432 192.168.1.100
#   2341 10.0.0.50
#   1234 172.16.0.1
#   ...
```

Example 2: Extract Data from CSV

```
# CSV file: id,name,department,salary
# Find all developers and their salaries

cat employees.csv \
  | grep -i "developer" \          # Filter developers
  | awk -F, '{print $2, $4}' \     # Extract name and salary
  | sort -k2 -rn                   # Sort by salary

# Count employees per department
awk -F, 'NR>1 {dept[$3]++} END {for (d in dept) print dept[d], d}' \
  employees.csv | sort -rn
```

Example 3: Config File Processing

```
# Update multiple config files at once
# Change old server IP 192.168.1.50 to new 192.168.2.100

find /etc/nginx/sites-available/ -name "*.conf" \
  -exec sed -i 's/192\.168\.1\.50/192.168.2.100/g' {} \;

# Verify the changes
grep -r "192.168" /etc/nginx/sites-available/
```

🔧 Hands-on Exercise 1.8

```
# Create test data
cat > /tmp/employees.txt << 'EOF'
Alice Developer 75000
Bob Manager 90000
Charlie Developer 80000
Diana Tester 65000
Eve Developer 78000
Frank Manager 95000
EOF

# 1. Find all Developers
grep "Developer" /tmp/employees.txt

# 2. How many developers are there?
grep -c "Developer" /tmp/employees.txt

# 3. Print just names and salaries
awk '{print $1, $3}' /tmp/employees.txt

# 4. What's the total salary of all developers?
grep "Developer" /tmp/employees.txt | awk '{sum += $3} END {print "Total:", sum}'

# 5. Sort by salary (ascending)
sort -k3 -n /tmp/employees.txt

# 6. Who earns the most?
sort -k3 -rn /tmp/employees.txt | head -1

# 7. Replace "Developer" with "Engineer"
sed 's/Developer/Engineer/g' /tmp/employees.txt
```

```
# 8. Count unique job titles
awk '{print $2}' /tmp/employees.txt | sort | uniq -c
```

Section 1.9: Process Management and Scheduling

🎯 Learning Objectives

- Understand what a process is and its lifecycle
- Read and interpret process information (PID, PPID, states)
- Manage process priority with nice and renice
- Monitor system resources with top and htop
- Schedule tasks with cron and at

Introduction

Every running program on Linux is a **process** — an instance of a program in execution. Even your terminal window is a process. At any given moment, your system has dozens to hundreds of processes running, each managed by the kernel's process scheduler.

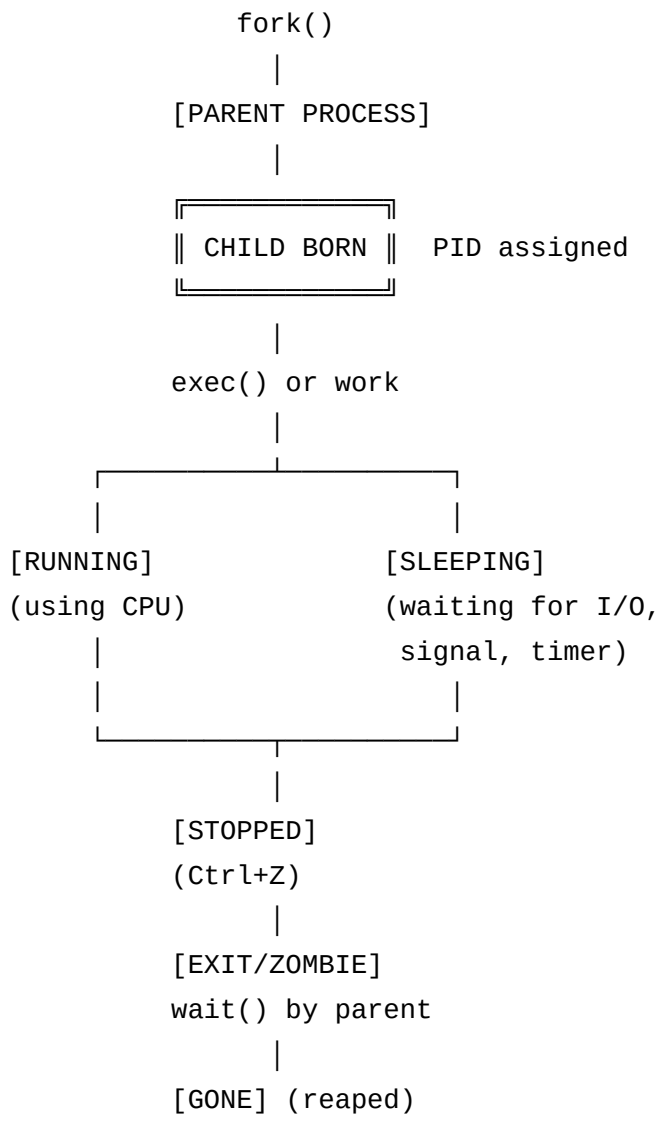
Understanding processes is crucial for troubleshooting performance issues, managing resources, and automating system tasks.

Processes and Their Attributes

Process Anatomy

```
A PROCESS HAS:
  PID   = Process ID (unique identifier, 1-32768 by default)
  PPID  = Parent Process ID (who spawned this process)
  UID   = User ID of process owner
  GID   = Group ID
  State = Current state (Running, Sleeping, etc.)
  Nice  = Priority (-20=highest to 19=lowest)
  VSZ   = Virtual memory size
  RSS   = Resident Set Size (physical RAM used)
  TTY   = Associated terminal (or ? for daemons)

PROCESS STATES:
  R = Running (actively using CPU)
  S = Sleeping (waiting for event, interruptible)
  D = Sleeping (uninterruptible, usually waiting for I/O)
  Z = Zombie (finished but parent hasn't read exit status)
  T = Stopped (suspended, e.g., by Ctrl+Z)
  I = Idle (kernel thread)
```

Process Lifecycle



Viewing Processes

```
# The basic process snapshot
ps aux
# USER  PID  %CPU %MEM  VSZ    RSS  TTY   STAT START   TIME COMMAND
# root    1   0.0  0.1 168736  14132 ?    Ss   Jan01   0:15 /sbin/init
# alice 2345  1.2  0.5 854321  82341 pts/0 Sl  10:30   0:05 /usr/bin/firefox

# Process tree (shows parent-child relationships)
pstree
pstree -p  # Include PIDs
pstree alice  # Only alice's processes

# More detailed process info
ps aux --sort=-%cpu | head -10   # Top CPU users
ps aux --sort=-%mem | head -10   # Top memory users

# Dynamic process viewer
top    # Press q to quit, h for help
htop   # Better visual (install with: sudo apt install htop)

# Find specific process
pgrep firefox     # Get PIDs by name
pgrep -a python   # Show command line too
pidof bash        # Another way
```

Process Signals (Revisited)

```
# Find and kill a misbehaving process
# Step 1: Find it
pgrep -a firefox
# or
ps aux | grep firefox

# Step 2: Try graceful kill first
kill <PID>           # SIGTERM — please stop
kill -15 <PID>       # Same as above

# Wait 5 seconds...

# Step 3: Force kill if needed
kill -9 <PID>        # SIGKILL — die now

# Kill by name
killall firefox      # All processes named 'firefox'
pkill -f "python myscript.py"  # Match full command line

# ⚠️ NEVER do: kill -9 1  (would kill init/systemd — crash!)
```

Process Scheduling and Prioritization

Understanding nice Values

```
PRIORITY SCALE:
  -20 ◄──────────────────────────────────► 19
  HIGHEST                              LOWEST
  (gets more CPU time)          (gets less CPU time)

DEFAULT nice value: 0
Only ROOT can set NEGATIVE nice values (increase priority)
Regular users can only INCREASE nice value (reduce priority)
```

```
# Start a process with specific priority
nice -n 10 python3 heavy_computation.py   # Low priority
nice -n -5 critical_task.sh               # Higher priority (need sudo)
sudo nice -n -20 very_important_task      # Highest priority

# Change priority of RUNNING process
renice -n 15 -p 12345     # Change PID 12345 to nice 15
renice -n 5 -u alice      # Change all alice's processes to nice 5
sudo renice -n -10 -p 12345  # Increase priority (needs sudo)

# See current priorities in top
top   # Look at 'NI' column (nice value) and 'PR' column (priority)

# See nice value of a process
ps -o pid,nice,comm -p 12345
```

Scheduling with cron

`cron` is the classic task scheduler for Linux. It runs tasks on a schedule automatically.

```
CRON SYNTAX:
  ┌──────────── minute (0-59)
  │ ┌──────────── hour (0-23)
  │ │ ┌──────────── day of month (1-31)
  │ │ │ ┌──────────── month (1-12)
  │ │ │ │ ┌──────────── day of week (0-7, both 0 and 7 = Sunday)
  │ │ │ │ │
  * * * * * command_to_run

SPECIAL VALUES:
  *  = Every possible value
  */5 = Every 5 units (every 5 minutes if in minute column)
  1-5 = Range (1 through 5)
  1,3,5 = List (1, 3, and 5)
```

```
# Edit your crontab (cron table)
crontab -e

# List your crontab
crontab -l

# Remove crontab
crontab -r

# EXAMPLES IN CRONTAB:
# Run at 6:30 AM every day
30 6 * * * /home/alice/morning_backup.sh

# Run every 5 minutes
*/5 * * * * /scripts/check_disk.sh

# Run at midnight on the first of every month
0 0 1 * * /scripts/monthly_report.sh

# Run at 9 AM on weekdays (Mon-Fri)
0 9 * * 1-5 /scripts/daily_standup_reminder.sh

# Run every hour
0 * * * * /scripts/check_services.sh

# Run at 11 PM on Sundays
0 23 * * 0 /scripts/weekly_backup.sh

# Log output
*/5 * * * * /scripts/monitor.sh >> /var/log/monitor.log 2>&1
```

Scheduling with at (One-time tasks)

```
# Schedule a one-time task
at 10:30 AM tomorrow << 'EOF'
/scripts/maintenance.sh
EOF

at now + 2 hours < script.sh    # Run in 2 hours
at 2024-12-31 23:59 < newyear.sh  # At specific date/time

# See pending jobs
atq
# 3       Thu Jan 18 10:30:00 2024 a alice

# Remove a job
atrm 3   # Remove job #3
```

Practical Examples

Example 1: Finding and Fixing a High-CPU Process

```
# Scenario: System is slow. Find the culprit.

# Method 1: top
top
# Look at top processes by CPU (sorted by default)
# Press 'P' to sort by CPU
# Press 'M' to sort by memory
# Press 'k' to kill (enter PID)

# Method 2: ps
ps aux --sort=-%cpu | head -5
# USER   PID  %CPU  %MEM   VSZ   RSS  TTY  STAT START  TIME COMMAND
# alice 9876  98.5   2.1 450000 34000 pts/0 R   10:30  5:23 python3 crawler.py

# Found it! PID 9876 using 98.5% CPU

# Option 1: Reduce its priority instead of killing
sudo renice -n 19 -p 9876   # Push to lowest priority

# Option 2: Suspend and investigate
kill -STOP 9876   # Pause the process
# ... investigate ...
kill -CONT 9876   # Resume
# or
kill -9 9876      # Kill it completely
```

Example 2: System Monitoring Script (using cron)

```
#!/bin/bash
# /scripts/monitor.sh
# Run every 5 minutes via cron

LOGFILE="/var/log/system_monitor.log"
THRESHOLD=90  # Alert if usage above 90%

# Get CPU usage
CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)

# Get memory usage
MEM=$(free | grep Mem | awk '{printf "%.0f", $3/$2 * 100}')

# Get disk usage
DISK=$(df / | tail -1 | awk '{print $5}' | cut -d'%' -f1)

TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

echo "$TIMESTAMP CPU:${CPU}% MEM:${MEM}% DISK:${DISK}%" >> $LOGFILE

# Alert if CPU or memory high
if [ "$CPU" -gt "$THRESHOLD" ] || [ "$MEM" -gt "$THRESHOLD" ]; then
    echo "$TIMESTAMP ALERT: CPU=${CPU}% MEM=${MEM}%" | mail -s "System Alert" admin@example.com
fi


# Add to cron
crontab -e
# Add: */5 * * * * /scripts/monitor.sh
```

🔧 Hands-on Exercise 1.9

```
# Exercise: Process Management

# 1. Find your shell's PID
echo "My bash PID: $$"

# 2. See the process tree from your shell
pstree -p $$

# 3. Start some background processes
sleep 100 &
sleep 200 &
sleep 300 &

# 4. List only YOUR background processes
ps aux | grep "sleep" | grep -v grep

# 5. Change priority of sleep 100 (get its PID first)
SLEEP_PID=$(pgrep -n sleep)  # Gets newest sleep PID
renice -n 10 -p $SLEEP_PID
ps -o pid,nice,comm -p $SLEEP_PID

# 6. Kill all sleep processes
pkill sleep
jobs

# 7. Create a simple cron job that runs every minute
crontab -e
# Add: * * * * * echo "tick" >> /tmp/cron_test.log

# Wait 2 minutes, then check:
```

```
# cat /tmp/cron_test.log

# 8. Remove the cron job
crontab -r
```

Section 1.10: Package Management

🎯 Learning Objectives

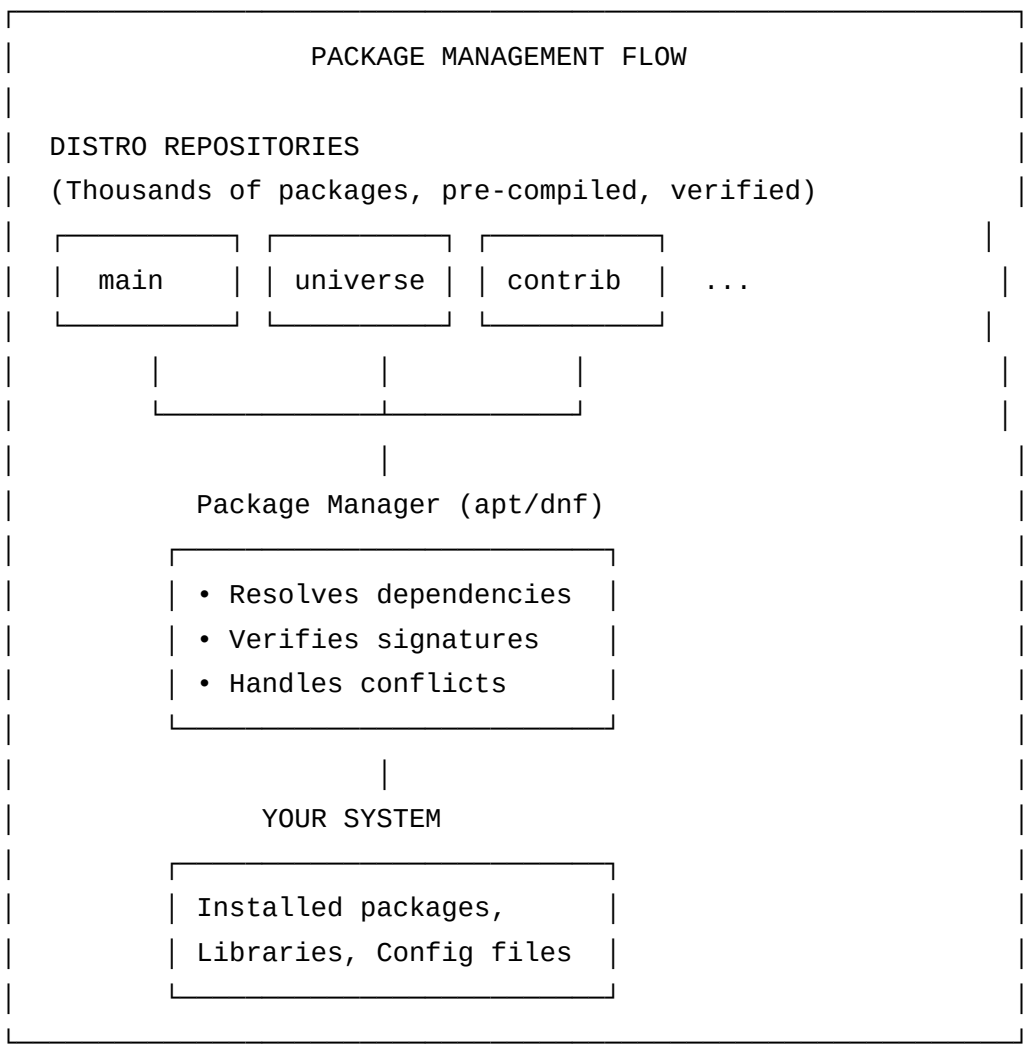
- Use apt (Debian/Ubuntu) for package management
- Use dnf/yum (Fedora/RHEL/CentOS) for package management
- Search, install, update, and remove packages
- Manage repositories
- Understand package dependencies

Introduction

Installing software on Linux is dramatically simpler than on Windows or macOS. Instead of visiting websites, downloading installers, clicking through wizards, and hoping for no malware — you type one command. The package manager handles downloading from trusted repositories, verifying signatures, resolving dependencies, and keeping everything updated.

Think of a package manager as an "app store" for the terminal, but where all apps are free and open source.

Package Management Overview



apt — Debian/Ubuntu Package Management

```
# ──────────────────────────────────────────────
# UPDATING
# ──────────────────────────────────────────────

# Update package index (list of available packages)
sudo apt update

# Upgrade installed packages
sudo apt upgrade

# Upgrade + handle changed dependencies
sudo apt full-upgrade

# Do both (update first, then upgrade)
sudo apt update && sudo apt upgrade -y

# ──────────────────────────────────────────────
# SEARCHING
# ──────────────────────────────────────────────

# Search for a package
apt search nginx

# Show detailed info about a package
apt show nginx

# List installed packages
apt list --installed

# List installed packages matching pattern
```

```
apt list --installed | grep python

# ──────────────────────────────────────────────
# INSTALLING
# ──────────────────────────────────────────────

# Install a package
sudo apt install nginx

# Install multiple packages
sudo apt install nginx php-fpm mysql-server

# Install without prompting
sudo apt install -y nginx

# Install specific version
sudo apt install nginx=1.18.0-0ubuntu1

# Install a .deb file
sudo dpkg -i package.deb
sudo apt install -f  # Fix any broken dependencies

# ──────────────────────────────────────────────
# REMOVING
# ──────────────────────────────────────────────

# Remove package (keep config files)
sudo apt remove nginx

# Remove package AND config files
sudo apt purge nginx
```

```
# Remove unused dependencies
sudo apt autoremove

# Clean download cache
sudo apt clean        # Remove all cached packages
sudo apt autoclean    # Remove outdated cached packages

# ──────────────────────────────────────────────
# INFORMATION
# ──────────────────────────────────────────────

# See files installed by a package
dpkg -L nginx

# Which package provides a file?
dpkg -S /usr/bin/python3

# Verify package integrity
dpkg -V nginx
```

dnf/yum — Fedora/RHEL/CentOS Package Management

```
# dnf is the modern replacement for yum (same commands, mostly)

# Update package index and upgrade
sudo dnf update

# Search
dnf search nginx
dnf info nginx

# Install
sudo dnf install nginx
sudo dnf install -y nginx      # No prompt
sudo dnf install *.rpm         # Install local RPM file

# Remove
sudo dnf remove nginx
sudo dnf autoremove            # Remove unused deps

# List installed
dnf list installed
dnf list installed | grep python

# History (very useful for rollbacks!)
dnf history
sudo dnf history undo 15       # Undo transaction #15

# Groups (install related packages together)
dnf grouplist
sudo dnf groupinstall "Development Tools"
```

pacman — Arch Linux

```
# Update and upgrade everything
sudo pacman -Syu

# Install package
sudo pacman -S nginx

# Remove package
sudo pacman -R nginx
sudo pacman -Rs nginx     # Also remove orphaned dependencies

# Search
pacman -Ss nginx          # Search repos
pacman -Qs nginx          # Search installed

# List installed
pacman -Q
pacman -Qi nginx          # Info about installed package

# AUR (Arch User Repository) with yay helper
yay -S package-name       # Install from AUR
```

Repository Management

```
# UBUNTU: Add a PPA (Personal Package Archive)
sudo add-apt-repository ppa:nginx/stable
sudo apt update
sudo apt install nginx

# Remove a PPA
sudo add-apt-repository --remove ppa:nginx/stable

# See enabled repositories
cat /etc/apt/sources.list
ls /etc/apt/sources.list.d/

# FEDORA: Add a repository
sudo dnf config-manager --add-repo https://rpm.example.com/repo.repo

# Enable/disable repos
sudo dnf config-manager --set-enabled repo-name
sudo dnf config-manager --set-disabled repo-name

# List repos
dnf repolist
```

Practical Examples

Example 1: Setting Up a LAMP Stack

```
# Linux + Apache + MySQL + PHP

# Ubuntu/Debian
sudo apt update
sudo apt install apache2 mysql-server php php-mysql php-common -y

# Start services
sudo systemctl start apache2
sudo systemctl start mysql
sudo systemctl enable apache2  # Start on boot
sudo systemctl enable mysql

# Secure MySQL
sudo mysql_secure_installation

# Test PHP
echo "<?php phpinfo(); ?>" | sudo tee /var/www/html/info.php
# Visit http://localhost/info.php in browser
```

Example 2: Keeping System Updated

```
# Create an auto-update script (run via cron)
cat > /scripts/auto_update.sh << 'EOF'
#!/bin/bash
LOGFILE="/var/log/apt_updates.log"
echo "=== Update: $(date) ===" >> $LOGFILE
apt update -q >> $LOGFILE 2>&1
apt upgrade -y -q >> $LOGFILE 2>&1
apt autoremove -y -q >> $LOGFILE 2>&1
echo "Done" >> $LOGFILE
EOF
chmod +x /scripts/auto_update.sh

# Schedule to run Sunday at 3 AM
echo "0 3 * * 0 /scripts/auto_update.sh" | sudo crontab -
```

Example 3: Finding Which Package Provides a Command

```
# You need the 'tree' command but don't know the package name

# Method 1: apt-file (install first)
sudo apt install apt-file
sudo apt-file update
apt-file search /usr/bin/tree
# tree: /usr/bin/tree

# Method 2: dpkg -S (for already-installed packages)
dpkg -S $(which python3)
# python3-minimal: /usr/bin/python3

# Method 3: Ubuntu-specific
sudo apt install command-not-found
sudo update-command-not-found
tree   # This will suggest: sudo apt install tree
```

🔧 Hands-on Exercise 1.10

```
# Exercise: Package Management Practice

# 1. Update package lists
sudo apt update

# 2. See how many upgradeable packages you have
apt list --upgradeable 2>/dev/null | wc -l

# 3. Search for 'htop'
apt search htop

# 4. See what htop contains before installing
apt show htop

# 5. Install htop
sudo apt install htop -y

# 6. Verify installation
which htop
htop --version

# 7. See what files htop installed
dpkg -L htop

# 8. Remove htop but keep config
sudo apt remove htop

# 9. Verify it's gone
which htop  # Should show nothing
```

```
# 10. Clean up orphaned packages
sudo apt autoremove -y
```

Section 1.11: Init Systems and Boot Process

🎯 Learning Objectives

- Trace the Linux boot sequence from power button to login prompt
- Understand BIOS/UEFI, bootloader, and init system roles
- Manage services with systemd
- Create and modify systemd unit files
- Troubleshoot boot issues

Introduction

Understanding the boot process is like understanding how a car starts — from the key turn to the engine running. When you press the power button on a Linux machine, a carefully choreographed sequence of events happens in milliseconds to bring the system from nothing to a fully functional environment. Knowing this process helps you configure startup behavior, troubleshoot boot failures, and manage system services.

Linux Boot Process

```
BOOT SEQUENCE OVERVIEW:
═══════════════════════════════════════════════════════

STEP 1: POWER ON
  CPU starts executing at a fixed memory address

STEP 2: BIOS/UEFI FIRMWARE
  ├── POST: Power-On Self-Test (test hardware)
  ├── Initialize CPU, RAM, storage
  └── Find boot device (disk, USB, network)

STEP 3: BOOTLOADER (GRUB2)
  ├── Loaded from boot sector / ESP partition
  ├── Shows boot menu (if multiple kernels/OS)
  ├── Loads Linux kernel (vmlinuz) into RAM
  └── Loads initial RAM disk (initrd/initramfs)

STEP 4: KERNEL INITIALIZATION
  ├── Decompress itself
  ├── Initialize core subsystems (CPU, memory, interrupts)
  ├── Mount initramfs as temporary root filesystem
  ├── Run /init from initramfs
  ├── Load kernel modules for storage drivers
  ├── Mount real root filesystem
  └── Hand control to init system (PID 1)

STEP 5: INIT SYSTEM (systemd)
  ├── PID 1 — everything else descends from this
  ├── Mount filesystems (/etc/fstab)
  ├── Start system services (network, logging, etc.)
```

```
  ├── Start display manager (GUI login screen)
  └── Present login prompt

STEP 6: USER LOGIN
  └── User logs in → shell or desktop session starts
```

Detailed Boot Timeline

```
# See how long each step took
systemd-analyze
# Startup finished in 4.217s (firmware) + 2.681s (loader)
#                          + 1.323s (kernel) + 8.451s (userspace)
# = 16.672s

# See which services took the most time
systemd-analyze blame | head -20
# 4.532s NetworkManager-wait-online.service
# 2.156s apt-daily-upgrade.service
# 1.234s mysql.service
# ...

# Graphical view (opens in browser)
systemd-analyze plot > boot-analysis.svg
```

GRUB2 — The Bootloader

```
# GRUB configuration
cat /etc/default/grub

# Key settings:
# GRUB_DEFAULT=0              # Default OS to boot (0=first)
# GRUB_TIMEOUT=5              # Show menu for 5 seconds
# GRUB_CMDLINE_LINUX=""       # Kernel parameters for all boots
# GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"  # Normal boot params

# After changing /etc/default/grub, update GRUB:
sudo update-grub   # Debian/Ubuntu
sudo grub2-mkconfig -o /boot/grub2/grub.cfg  # Fedora/RHEL

# Common GRUB kernel parameters:
# quiet      = Suppress most boot messages
# splash     = Show splash screen
# nomodeset  = Disable kernel mode setting (GPU troubleshooting)
# single     = Boot into single-user mode (recovery)
# 3          = Boot to runlevel 3 (no GUI)
```

Init Systems

systemd (Modern Standard)

```
SYSTEMD ARCHITECTURE:
═══════════════════════════════════════════

  systemd (PID 1)
  │
  ├── Targets (like runlevels)
  │   ├── sysinit.target      (hardware init)
  │   ├── basic.target        (basic system)
  │   ├── network.target      (networking up)
  │   ├── multi-user.target   (no GUI, text mode)
  │   └── graphical.target    (with GUI desktop)
  │
  ├── Services (.service units)
  │   ├── sshd.service
  │   ├── nginx.service
  │   ├── mysql.service
  │   └── ...hundreds more
  │
  ├── Mounts (.mount units)
  │   ├── /etc/fstab entries
  │   └── tmpfs, devtmpfs, etc.
  │
  └── Timers (.timer units) — cron replacement
═══════════════════════════════════════════
```

systemctl — The systemd Control Command

```
# ──────────────────────────────────────────
# SERVICE MANAGEMENT
# ──────────────────────────────────────────

# Start/stop/restart a service
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx
sudo systemctl reload nginx       # Reload config without full restart

# Enable/disable (control whether starts at boot)
sudo systemctl enable nginx       # Start at boot
sudo systemctl disable nginx      # Don't start at boot
sudo systemctl enable --now nginx # Enable AND start immediately

# Check status
systemctl status nginx
# ● nginx.service - A high performance web server
#    Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
#    Active: active (running) since Mon 2024-01-15 10:30:00 UTC
#   Process: 1234 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
#  Main PID: 1235 (nginx)
#    CGroup: /system.slice/nginx.service
#            ├─1235 nginx: master process
#            └─1236 nginx: worker process

# Is it running?
systemctl is-active nginx    # Prints: active or inactive
systemctl is-enabled nginx   # Prints: enabled or disabled
systemctl is-failed nginx    # Check if it failed
```

```
# ──────────────────────────────────────────
# LISTING SERVICES
# ──────────────────────────────────────────

# List all services
systemctl list-units --type=service

# List only running services
systemctl list-units --type=service --state=running

# List failed services
systemctl --failed

# List all unit files
systemctl list-unit-files | grep service

# ──────────────────────────────────────────
# SYSTEM STATE
# ──────────────────────────────────────────

# Reboot
sudo systemctl reboot

# Shutdown
sudo systemctl poweroff

# Suspend
sudo systemctl suspend

# Enter rescue mode (minimal single-user)
```

```
sudo systemctl rescue

# ──────────────────────────────────────────
# SYSTEM TARGETS (runlevels)
# ──────────────────────────────────────────

# See current target
systemctl get-default

# Change default target
sudo systemctl set-default multi-user.target    # No GUI
sudo systemctl set-default graphical.target     # With GUI

# Switch to target immediately
sudo systemctl isolate multi-user.target

# Target equivalents:
# runlevel 0 → poweroff.target
# runlevel 1 → rescue.target
# runlevel 3 → multi-user.target
# runlevel 5 → graphical.target
# runlevel 6 → reboot.target
```

Creating a Custom systemd Service

```
# Goal: Create a service that runs our backup script on startup

# Step 1: Create the service file
sudo nano /etc/systemd/system/mybackup.service

# CONTENT OF /etc/systemd/system/mybackup.service:
```

```
[Unit]
Description=My Backup Service
Documentation=https://example.com/backup
After=network.target mysql.service
Requires=mysql.service

[Service]
Type=oneshot
User=backup_user
Group=backup_group
WorkingDirectory=/opt/backup

# The main command to run
ExecStart=/opt/backup/run_backup.sh

# Run this on failure
ExecStopPost=/opt/backup/notify_failure.sh

# Environment variables
Environment=BACKUP_DIR=/var/backups
EnvironmentFile=/etc/backup.conf

# Restart behavior
Restart=on-failure
RestartSec=30

# Resource limits
LimitNOFILE=65536
MemoryMax=512M
```

```
# Logging
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target


# Step 2: Reload systemd daemon (so it knows about the new service)
sudo systemctl daemon-reload

# Step 3: Enable and start
sudo systemctl enable mybackup.service
sudo systemctl start mybackup.service

# Step 4: Check it's working
systemctl status mybackup.service
journalctl -u mybackup.service -f   # Follow its logs
```

journalctl — Viewing systemd Logs

```
# All logs (most recent first: use --reverse or -r)
journalctl

# Follow new logs in real time
journalctl -f

# Logs from a specific service
journalctl -u nginx
journalctl -u nginx -f    # Follow

# Logs since last boot
journalctl -b

# Logs from previous boot
journalctl -b -1

# Logs since/until time
journalctl --since "2024-01-15 10:00:00"
journalctl --since "2 hours ago"
journalctl --since "yesterday" --until "today"

# Priority filter
journalctl -p err     # Only errors and worse
journalctl -p warning # Warnings and worse

# Kernel messages only
journalctl -k

# From a specific binary
journalctl /usr/bin/sshd
```

```
# Output format
journalctl -u nginx -o json | head -5   # JSON format
journalctl -u nginx --no-pager > /tmp/nginx.log  # To file
```

SysVinit (Legacy)

```
# Some older systems or containers still use SysVinit

# Start/stop services (SysVinit way)
sudo service nginx start
sudo service nginx stop
sudo service nginx restart
sudo service nginx status

# Enable at boot (SysVinit)
sudo update-rc.d nginx enable
sudo update-rc.d nginx disable

# Runlevels:
# 0 = Halt
# 1 = Single user mode
# 2 = Multi-user (without NFS)
# 3 = Full multi-user mode
# 4 = Unused
# 5 = Full multi-user with GUI
# 6 = Reboot

# Change runlevel
sudo init 3    # Switch to runlevel 3 (no GUI)
sudo telinit 3 # Same thing
```

Practical Examples

Example 1: Diagnosing a Boot Problem

```
# System is slow to boot — let's diagnose

# See overall boot time
systemd-analyze

# Find slow services
systemd-analyze blame | head -10

# Check for failed services
systemctl --failed
# UNIT                         LOAD   ACTIVE SUB    DESCRIPTION
# mysqld.service               loaded failed failed MySQL Server

# See why mysql failed
journalctl -u mysqld -b      # Logs from this boot
journalctl -u mysqld -b --no-pager | tail -30

# Common fix: check disk space (full disk breaks MySQL)
df -h
```

Example 2: Creating a Web App Service

```
# You have a Node.js app at /opt/webapp/app.js
# You want it to run as a system service

# Create service file
sudo tee /etc/systemd/system/webapp.service << 'EOF'
[Unit]
Description=My Node.js Web Application
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/opt/webapp
ExecStart=/usr/bin/node /opt/webapp/app.js
Restart=always
RestartSec=10
StandardOutput=syslog
StandardError=syslog
SyslogIdentifier=webapp
Environment=NODE_ENV=production PORT=3000

[Install]
WantedBy=multi-user.target
EOF

# Deploy
sudo systemctl daemon-reload
sudo systemctl enable --now webapp
sudo systemctl status webapp
```

```
# View logs
journalctl -u webapp -f
```

🔧 Hands-on Exercise 1.11

```
# Exercise: systemd Service Management

# 1. Check system boot time
systemd-analyze

# 2. See top 5 slowest boot services
systemd-analyze blame | head -5

# 3. Check status of SSH service
systemctl status sshd

# 4. Is SSH enabled at boot?
systemctl is-enabled sshd

# 5. List running services
systemctl list-units --type=service --state=running | head -10

# 6. Check for any failed services
systemctl --failed

# 7. See recent system logs (last 20 minutes)
journalctl --since "20 minutes ago"

# 8. Create a simple test service
sudo tee /etc/systemd/system/hello.service << 'EOF'
[Unit]
Description=Hello World Test Service

[Service]
Type=oneshot
```

```
ExecStart=/bin/echo "Hello from systemd!"
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl start hello
sudo systemctl status hello
journalctl -u hello --no-pager

# 9. Cleanup
sudo systemctl stop hello
sudo rm /etc/systemd/system/hello.service
sudo systemctl daemon-reload
echo "Cleanup done!"
```

APPENDIX A: GLOSSARY

TERM	DEFINITION
Bash	Bourne Again SHell - the most common Linux shell
Boot Loader	Software that loads the OS kernel (e.g., GRUB2)
Daemon	Background service process (e.g., sshd, nginx)
Distro	Distribution - a complete Linux OS package
ext4	Fourth extended filesystem - most common Linux FS
FOSS	Free and Open Source Software
GRUB2	GRand Unified Bootloader v2 - standard Linux bootloader
initramfs	Initial RAM filesystem - temporary FS during boot
Inode	Data structure storing file metadata
Kernel	Core of the OS - manages hardware and resources
LTS	Long-Term Support - maintained for many years
PID	Process ID - unique number for each running process
PTY	Pseudo-Terminal - software terminal emulation
Root	Superuser account with full system access (uid=0)
RPM	Red Hat Package Manager format
Shell	Command interpreter (bash, zsh, fish, etc.)
Signal	IPC mechanism to notify processes of events
stdout/stderr	Standard output/error streams
sudo	Superuser Do - run command with elevated privileges
Symlink	Symbolic Link - file pointing to another file
systemd	Modern init system and service manager
TTY	TeleTYpewriter - terminal device
UID/GID	User/Group ID numbers
VFS	Virtual File System - abstraction for all filesystems
vmlinuz	Compressed Linux kernel executable

APPENDIX B: QUICK REFERENCE CHEAT SHEETS

Navigation Cheat Sheet

```
pwd               Where am I?
cd /path          Go to path
cd ~              Go home
cd ..             Go up one level
cd -              Go to previous location
ls -lah           List files (long, all, human-readable)
find ~ -name "*.txt"  Find txt files
```

File Operations

```
cp -r src dst     Copy (recursive for dirs)
mv old new        Move/rename
rm -rf dir/       Delete directory (careful!)
touch file        Create empty file
mkdir -p a/b/c    Create directory tree
ln -s target link Create symlink
```

Permissions

`chmod 755 file`	`rwxr-xr-x`
`chmod 644 file`	`rw-r--r--`
`chmod 600 file`	`rw-------`
`chown user:group`	Change owner and group
`sudo`	Run as root

Process Management

`ps aux`	All processes
`top / htop`	Interactive process viewer
`kill PID`	Terminate process
`kill -9 PID`	Force kill
`Ctrl+C`	Kill foreground process
`Ctrl+Z`	Suspend foreground
`bg/fg`	Background/foreground
`jobs`	List background jobs
`nohup cmd &`	Run after logout

Text Processing

```
grep pattern file  Search for pattern
grep -r pat dir/   Recursive search
sed 's/old/new/g'  Replace text
awk '{print $2}'   Extract column
sort file          Sort lines
uniq               Remove duplicates
wc -l              Count lines
head -20           First 20 lines
tail -f            Follow file
```

Package Management (Ubuntu)

```
sudo apt update    Refresh package list
sudo apt upgrade   Upgrade packages
sudo apt install X Install package X
sudo apt remove X  Remove package X
apt search X       Search for package
apt show X         Package info
dpkg -L X          Files installed by X
```

systemd/Services

```
systemctl start/stop/restart nginx
systemctl enable/disable nginx
systemctl status nginx
systemctl list-units --type=service
journalctl -u nginx -f
journalctl -b         Logs from this boot
systemd-analyze       Boot time analysis
```

APPENDIX C: INTERVIEW QUESTIONS

Beginner Level

1. What is the difference between Linux and GNU/Linux?
2. What does PID stand for and what is special about PID 1?
3. Explain the permission string rwxr-xr-x.
4. What is the purpose of /etc/passwd?
5. What does `ls -la` show that `ls` doesn't?
6. Explain the difference between soft and hard links.
7. What is a daemon process?

Intermediate Level

8. What is the difference between `kill -9` and `kill -15` ?

9. How does `sudo` work? What file controls it?
10. Explain the difference between ext4 and XFS.
11. What is the init system and why was systemd controversial?
12. How do you check why a service failed to start?
13. What is the difference between `apt remove` and `apt purge` ?
14. Explain the Linux boot process.

Advanced Level

15. Explain virtual memory and the purpose of swap.
16. What is the difference between user space and kernel space?
17. How does the VFS abstraction benefit Linux?
18. What are orphan and zombie processes?
19. Explain the purpose of the sticky bit on /tmp.
20. How would you debug a system that is slow to boot?
21. What are cgroups and namespaces, and why are they important for containers?
22. Explain the Linux networking stack layers.

Answers Summary (Key Points)

- **PID 1**: init/systemd — all other processes descend from it
- **rwxr-xr-x = 755**: owner=rwx, group=r-x, others=r-x
- **kill -9 vs -15**: -15 (SIGTERM) polite request; -9 (SIGKILL) forced, cannot be caught
- **sudo**: Checks /etc/sudoers, verifies password, runs command as root
- **Zombie**: Process that died but parent hasn't called wait() to get exit status

APPENDIX D: CERTIFICATION GUIDE

Relevant Certifications

CERTIFICATION	PROVIDER	LEVEL	FOCUS
LPIC-1	LPI	Beginner	Linux fundamentals
LPIC-2	LPI	Intermediate	Admin
LPIC-3	LPI	Advanced	Specialized
CompTIA Linux+	CompTIA	Beginner/Int	All-round Linux
RHCSA	Red Hat	Intermediate	RHEL admin
RHCE	Red Hat	Advanced	RHEL engineer
CKA	CNCF	Advanced	Kubernetes (Linux based)
LFCS	Linux Fdn	Intermediate	System Admin

Study Resources

- **Linux Foundation free courses:** training.linuxfoundation.org
- **Red Hat learning:** www.redhat.com/en/services/training
- **Linux Professional Institute:** lpi.org
- **Practice exams:** examtopics.com, udemy.com

APPENDIX E: PROJECT IDEAS

Project 1: System Monitoring Dashboard

Difficulty: Beginner
Skills: Shell scripting, text processing, cron

Create a script that collects CPU, memory, disk, and network statistics every 5 minutes and stores them in CSV format. Then create an HTML report showing trends.

```
# Starter code
#!/bin/bash
LOGFILE="/var/log/system_stats.csv"
echo "timestamp,cpu_pct,mem_pct,disk_pct" > $LOGFILE

collect_stats() {
    local ts=$(date +%s)
    local cpu=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | tr -d '%')
    local mem=$(free | awk '/Mem/ {printf "%.0f", $3/$2*100}')
    local disk=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
    echo "$ts,$cpu,$mem,$disk" >> $LOGFILE
}

collect_stats
```

Project 2: Automated Backup System

Difficulty: Intermediate
Skills: Shell scripting, cron, tar, rsync

Build a backup system that:

- Creates daily, weekly, and monthly backups
- Rotates old backups automatically
- Sends email notifications on success/failure
- Tests backup integrity

Project 3: User Account Manager

Difficulty: Intermediate
Skills: User management, bash scripting

Create a script that manages user accounts with:

- Add/delete/modify users from a CSV file
- Set up SSH keys automatically
- Apply group memberships
- Generate audit reports

Project 4: Web Server Setup Automation

Difficulty: Intermediate
Skills: Package management, systemd, networking

Write a script that:

- Installs nginx + PHP + MySQL
- Creates a virtual host for a given domain
- Sets up SSL certificates (with Let's Encrypt)
- Configures firewall rules
- Creates initial database and user

Project 5: Log Analysis Tool

Difficulty: Advanced
Skills: awk, sed, grep, text processing

Build a log analysis tool that:

- Parses Apache/nginx access logs
- Identifies top IPs, URLs, error codes
- Detects suspicious patterns (brute force, scanners)
- Generates HTML reports

APPENDIX F: TROUBLESHOOTING GUIDE

Boot Problems

Symptom: System won't boot, drops to emergency shell

```
# Check filesystem errors
fsck /dev/sda1    # Check filesystem (NOT while mounted!)
# In emergency shell: journalctl -xb to see what failed
# Common fix: Corrupted /etc/fstab
# Boot from live USB, mount partition, fix /etc/fstab
```

Symptom: GRUB error / can't find kernel

```
# Boot from live USB
# Mount your partition: sudo mount /dev/sda1 /mnt
# Chroot: sudo chroot /mnt
# Reinstall grub: grub-install /dev/sda
# Update grub: update-grub
```

Permission Problems

Symptom: Permission denied errors

```
# Check file permissions
ls -la problematic_file

# Check if you need sudo
sudo command

# Check directory permissions (need execute to traverse)
ls -la /path/to/directory

# Check SELinux/AppArmor if enabled
ls -laZ file    # See SELinux context
sudo aa-status  # AppArmor status
```

Service Problems

Symptom: Service fails to start

```
# Check what happened
systemctl status servicename
journalctl -u servicename -b --no-pager

# Common causes:
# - Port already in use: ss -tlnp | grep PORT
# - Permission denied: check log files, config ownership
# - Syntax error in config: service specific check command
# - Missing dependency: check "After=" and "Requires=" in unit file
```

Disk Space Problems

Symptom: No space left on device

```
# Find what's taking space
du -sh /* 2>/dev/null | sort -rh | head -20
du -sh /var/log/* | sort -rh | head -10

# Clean up logs
sudo journalctl --vacuum-size=100M    # Keep only 100MB of logs
sudo apt autoremove && sudo apt clean  # Clean package cache

# Find large files
find / -type f -size +1G 2>/dev/null
```

Network Problems

Symptom: No network connectivity

```
# Check interface status
ip addr show
ip link show

# Check routing
ip route show

# Test connectivity
ping 8.8.8.8                    # Test internet
ping $(hostname)                 # Test local
traceroute 8.8.8.8              # Trace path

# Check DNS
cat /etc/resolv.conf
nslookup google.com
dig google.com

# Restart networking
sudo systemctl restart NetworkManager
```

APPENDIX G: NEXT STEPS — WHAT TO LEARN AFTER THIS GUIDE

Congratulations on completing this Linux System Programming guide! Here's your roadmap for continued learning:

Immediate Next Steps (Months 2-3)

- **Bash Scripting Mastery:** Learn functions, arrays, error handling
- **Networking Deep Dive:** TCP/IP, DNS, firewalls (iptables/nftables)
- **Security Hardening:** SSH configuration, fail2ban, SELinux/AppArmor

Intermediate Goals (Months 4-6)

- **Linux Performance Tuning:** perf, strace, ltrace, system tracing
- **Containerization:** Docker and Docker Compose
- **Configuration Management:** Ansible

Advanced Topics (6+ Months)

- **Kubernetes:** Container orchestration
- **Linux Kernel Module Development:** Write your own kernel code
- **Systems Programming in C:** glibc, system calls, IPC

Recommended Books

1. "The Linux Command Line" — William Shotts
2. "How Linux Works" — Brian Ward
3. "Linux System Programming" — Robert Love
4. "The Linux Programming Interface" — Michael Kerrisk
5. "Linux Kernel Development" — Robert Love

Online Resources

- **Linux Kernel Documentation:** kernel.org/doc
- **Arch Wiki:** wiki.archlinux.org (excellent for all distros)
- **DigitalOcean Tutorials:** digitalocean.com/community/tutorials
- **Linux Journey:** linuxjourney.com
- **Explainshell:** explainshell.com (explains any command)

This guide was created as part of a 40-day intensive Linux System Programming curriculum. The journey from zero to mastery in Linux is challenging but extraordinarily rewarding. The command line gives you superpowers — use them wisely.

Happy Hacking! 🐧